

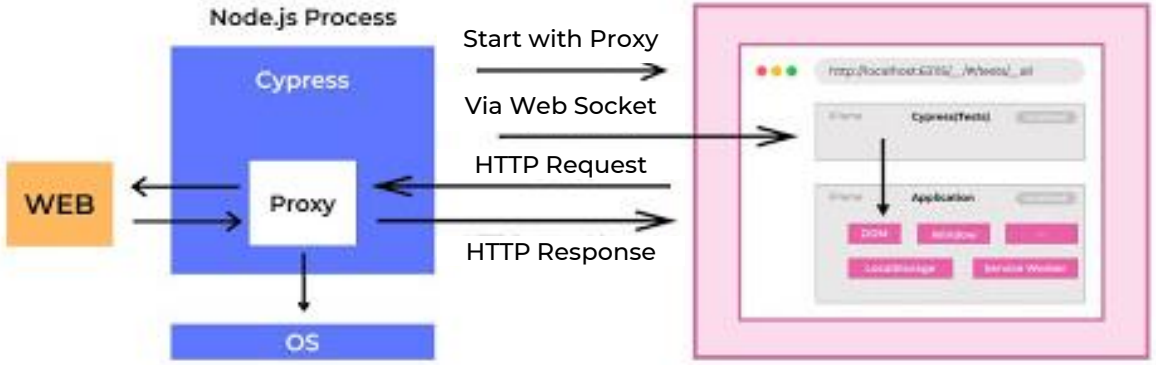
Week 14: Testing & Deployment

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

Cypress Basics

- **Cypress เป็น frontend testing tool สำหรับ web applications**
  - เป็น open-source test automation framework สำหรับ test JavaScript web
  - รองรับการทำ unit testing, integration testing ไปจนถึง end-to-end tests
  - Cypress automation testing เริ่ม execution process บน NodeJS server ที่สื่อสารกับ test runner เพื่อ run application และ test code
  - ทั้ง automated code และ application code แชร์ platform เดียวกัน ทำให้ developer ควบคุม application ที่กำลังได้รับการ test ได้โดยสะดวก



Cypress Basics



- Developer สามารถเขียน Cypress Test ได้ โดยสร้างเป็น Test Case ซึ่งจะถูก run
  - ตัวอย่าง

```
describe('Simple tests', () => {
  it('Visit Testomat.io website', () => {
    cy.visit('https://testomat.io/');
  });
});
```

cy.visit() สำหรับ automate การ visit URL ที่ระบุ

- Cypress Query สำหรับเข้าถึง Element
  - cy.get(…) เข้าถึง DOM Elements โดย selector หรือ alias

```
cy.get(selector)
cy.get(alias)
cy.get(selector, options)
cy.get(alias, options)
```

  - เข้าถึง Element li ใน Element ที่มีคลาส .list

```
cy.get('.list > li')
```

Cypress Basics



- `cy.get( … )` เข้าถึง DOM Elements
  - เข้าถึง input element

```
cy.get('input')
```

  - เข้าถึง link ที่มี `href` attribute บรรจุคำว่า "questions" แล้วทำการ click

```
cy.get('a[href*="questions"]').click()
```

- `.as()` ใช้ในการสร้าง Alias

```
.as('aliasName')
```

ตัวอย่าง

```
cy.get('.main-nav').find('li').first().as('firstNav')
```

- สามารถเข้าถึงสิ่งที่สร้าง alias ไว้ ผ่านการ Query ด้วย

```
cy.get('@aliasName')
```

Cypress Basics



- **Cypress Assertion** ใช้ assertion จาก Chai assertion library
  - รองรับ assertion ด้วย `expect`/`should`
  - ตัวอย่าง

```
cy.get('input').should('be.disabled')
```

```
cy.get('[data-test-id="test-example"]').should('have.length', 5)
```

  - ตัวอย่างการทำ Assertion https://docs.cypress.io/guides/references/assertions

exist	`.should('exist')` `expect(myVar).to.exist`
empty	`.should('be.empty')` `expect([]).to.be.empty`
arguments **Aliases:** Arguments	`.should('be.arguments')` `expect(arguments).to.be.arguments`
equal(*value*) **Aliases:** equals, eq	`.should('equal', 42)` `expect(42).to.equal(42)`

Cypress Basics

- ตัวอย่างการทำ Assertion

any	`.should('have.any.keys', 'age')` `expect(arr).to.have.any.keys('age')`
all	`.should('have.all.keys', 'name', 'age')` `expect(arr).to.have.all.keys('name', 'age')`
a(*type*) **Aliases:** an	`.should('be.a', 'string')` `expect('test').to.be.a('string')`
include(*value*) **Aliases:** contain, includes, contains	`.should('include', 2)` `expect([1,2,3]).to.include(2)`
ok	`.should('not.be.ok')` `expect(undefined).to.not.be.ok`
true	`.should('be.true')` `expect(true).to.be.true`
false	`.should('be.false')` `expect(false).to.be.false`
null	`.should('be.null')` `expect(null).to.be.null`

Cypress Basics

- ตัวอย่างการทำ Assertion

not	`.should('not.equal', 'Jane')` `expect(name).to.not.equal('Jane')`
deep	`.should('deep.equal', { name: 'Jane' })` `expect(obj).to.deep.equal({ name: 'Jane' })`

nested	`.should('have.nested.property', 'a.b[1]')`
	`.should('nested.include', {'a.b[1]': 'y'})`
	`expect({a: {b: 'x'}}).to.have.nested.property('a.b')`
	`expect({a: {b: 'x'}}).to.nested.include({'a.b': 'x'})`

ordered	`.should('have.ordered.members', [1, 2])`
	`expect([1, 2]).to.have.ordered.members([1, 2])`
	`expect([1, 2]).not.to.have.ordered.members([2, 1])`

eql(*value*) **Aliases:** eqls	`.should('eql', { name: 'Jane' })` `expect({ name: 'Jane' }).to.eql({ name: 'Jane' })`
greaterThan(*value*) **Aliases:** gt, above	`.should('be.greaterThan', 5)` `expect(10).to.be.greaterThan(5)`
least(*value*) **Aliases:** gte	`.should('be.at.least', 10)` `expect(10).to.be.at.least(10)`
lessThan(*value*) **Aliases:** lt, below	`.should('be.lessThan', 10)` `expect(5).to.be.lessThan(10)`

Cypress Basics



- **Cypress Query** สำหรับเข้าถึง Element
  - `cy.find( … )` เข้าถึง DOM Elements ที่เป็น descendent โดย selector

```
.find(selector)
.find(selector, options)
```

- หา `footer` ใน `.article`

```
cy.get('.article').find('footer')
```

- หา `li` ใน `parent`

```
<ul id="parent">
  <li class="first"></li>
  <li class="second"></li>
</ul>
```

```
cy.get('#parent').find('li')
```

Cypress Basics



- **Cypress Query** สำหรับเข้าถึง Element
  - `cy.contains( … )` เข้าถึง DOM Elements ที่บรรจุ text

```
.contains(content)
.contains(content, options)
.contains(selector)
.contains(selector, options)
```

- เข้าถึง element แรกใน document ที่บรรจุ Hello

```
cy.contains('Hello')
```

- เข้าถึง element ใน `.nav` ที่บรรจุ About

```
cy.get('.nav').contains('About')
```

- เข้าถึง element ใน `form` ที่บรรจุ submit the form! แล้ว click

```
cy.get('form').contains('submit the form!').click()
```

Cypress Basics



- Cypress มาพร้อม Hooks สำหรับ set condition ที่ต้องการ run ก่อน หรือ หลัง ชุด test หรือ แต่ละ test
  - `before()` hooks — Run ก่อนทุก Test
  - `beforeEach()` hooks — Run ก่อนแต่ละ Test
  - `after()` hooks — Run หลังทุก Test
  - `afterEach()` Hooks — Run หลังแต่ละ Test

```
describe('Hooks', () => {
  before(() => {
    // runs once before all tests in the block
  })

  beforeEach(() => {
    // runs before each test in the block
  })

  afterEach(() => {
    // runs after each test in the block
  })

  after(() => {
    // runs once after all tests in the block
  })
})
```

Cypress Basics



- ถ้าวางที่ระดับ Root ของ Test
  - `before()` hooks — Run ก่อนทุก Test
  - `beforeEach()` hooks — Run ก่อนแต่ละ Test Block
  - `after()` hooks — Run หลังจากเสร็จทุก Test
  - `afterEach()` Hooks — Run หลังแต่ละ Test Block

```
before(() => {
  // root-level hook
  // runs once before all tests
})

beforeEach(() => {
  // root-level hook
  // runs before every test block
})

afterEach(() => {
  // runs after each test block
})

after(() => {
  // runs once all tests are done
})

describe('Hooks', () => {
```

Cypress Basics



Cypress มาพร้อม Libraries ที่มี APIs สำหรับการทดสอบ application

- `cy.stub()` API สำหรับการสร้าง stub ซึ่งเป็น code ตัวแทน (Stand in) สำหรับชิ้น code อื่น
  - ทดแทน code ตัวจริง (เช่น Function, Module ต่างๆ) เพื่อ modify หรือ control พฤติกรรม ใช้สำหรับ testing นิยมใช้กำหนดค่าผลลัพธ์ด้วยค่าที่กำหนดไว้ล่วงหน้า
  - Stub ถูกเรียกใช้แทน code จริงเพื่อควบคุมพฤติกรรมของสิ่งที่ test เป็นการลด Dependencies
  - การสร้าง Standalone Stub

```
cy.stub()
```

  - การสร้าง Stub สำหรับเมธอด *obj.method()*

```
cy.stub(obj,'method',stubFunction)
```

    - *stubFunction* ฟังก์ชั่นที่จะเป็น Stub ให้กับ *obj.method()*

ตัวอย่าง

```
const winHrefStub = (url) => {
  expect(url).to.eq('tel://0234567890')
}
cy.window().then(win => {
  cy.stub(win, 'setLocationHref', winHrefStub)
})
```

Cypress Basics



- การสร้าง Stub สำหรับเมธอด *obj.method()* ทำการ return ค่าเป็น *'foo'*

```
cy.stub(obj,'method').returns('foo')
```

- การสร้าง Stub สำหรับเมธอด *obj.method()* เมื่อถูกเรียกด้วย argument *'bar'* ให้ return ค่าเป็น *'foo'*

```
cy.stub(obj,'method').withArgs('bar').returns('foo')
```

- การสร้าง Stub สำหรับเมธอด *obj.method()* ให้ return promise ที่ resolve ค่าเป็น *'foo'*

```
cy.stub(obj,'method').resolves('foo')
```

Cypress Basics



- `cy.spy()` API สำหรับการสร้าง code ที่ใช้ spy การเรียกชิ้น code อื่น
  - คล้ายกับ Stub แต่ไม่มีการ modify หรือ control พฤติกรรม แต่ใช้จับตาดู (spy on) การใช้งาน code อื่น (เช่น Function, Module ต่างๆ)
  - Capture และ Assert ดูว่า function นั้น ถูกเรียกให้ทำงานอย่างไร ตัวอย่างเช่น
    - ถูกเรียกใช้งานด้วย argument ที่ถูกต้อง หรือไม่
    - ถูกเรียกเป็นจำนวนครั้งที่ควรจะเป็นหรือไม่
    - ดูค่า return value
  - การสร้าง Spy สำหรับเมธอด *obj.method()*

```
cy.spy(obj,'method')
```

```
cy.window().then(win => {
   cy.spy(win,'confirm').as('winConfirm')
})
…
cy.get('@winConfirm').should('be.calledOnce')
```

Automate Option Selection



- Cypress ลองรับการ Automate การเลือก Dropdown List
  - Automate การเลือก option TH

```
cy.get('select').select('optionValue')
```

    - *optionValue* ค่า Value ของ option ที่จะเลือก
  - Automate การเลือกหลาย option

```
cy.get('select').select(['optionValue1', 'optionValue2',…])
```

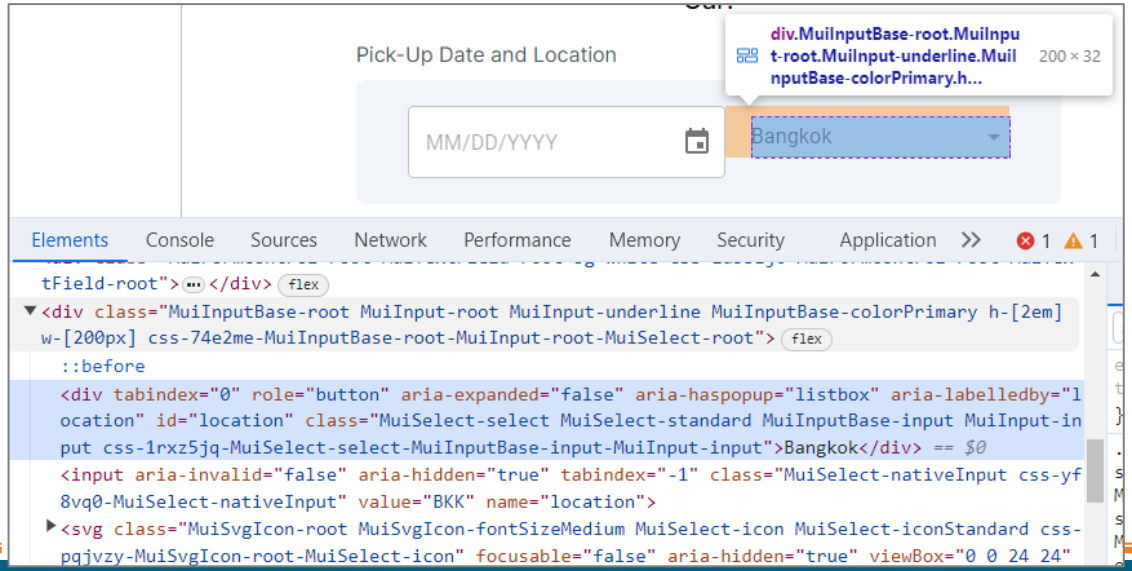
  - ตัวอย่าง การ Automate การเลือก และ assert ค่า

```
<select id="country">
<option value="TH">Thailand</option>
<option value="CN">China</option>
</select>
```

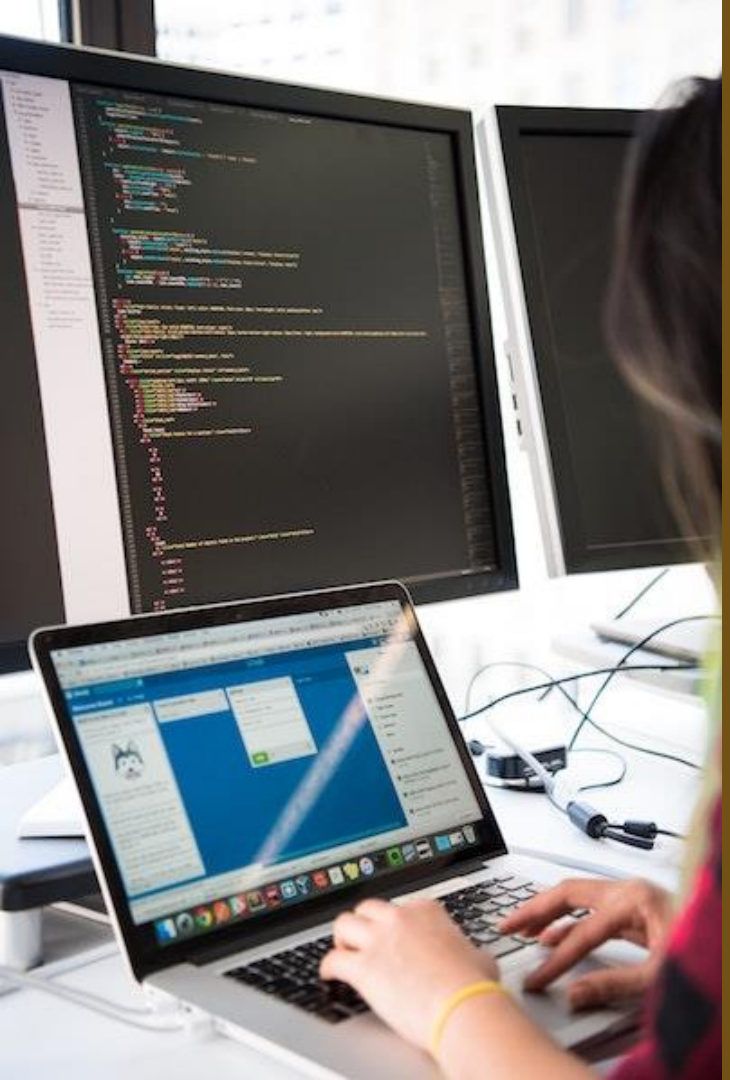
```
cy.get('select[id="country"]').select('TH').should('have.value','TH')
```

Automate MUI Option Selection

- `Select` Component ของ MUI เป็น `<div>...</div>`
  - ID ของ form label ควบคุมโดย `div` ตัวนอก การเลือกทำโดย
    - เข้าถึง label ตัวใน
    - Click parent component ของ label เพื่อเปิด dropdown
    - เข้าถึง item ใน dropdown bar ที่จะเลือกซึ่งเป็น element `ul` > `li` ที่มี attribute `data-value` เป็นค่าที่จะเลือก




```
cy.get('#selectID')
  .parent()
  .click()
  .get('ul>li[data-value="optionVal"]')
  .click();
```



Workshop 14.1: Using Cypress

Using Cypress



- ติดตั้ง Dev Dependencies Package (`-D` หรือ `--save-dev`) ของ Cypress Version 13.6.3

```
npm i -D cypress
```

- สังเกต Dev Dependencies และ version ที่สร้างขึ้นมาใน `package.json`
- ใน `package.json` เพิ่ม script สำหรับการ run npm ให้ลองรับการใช้ cypress

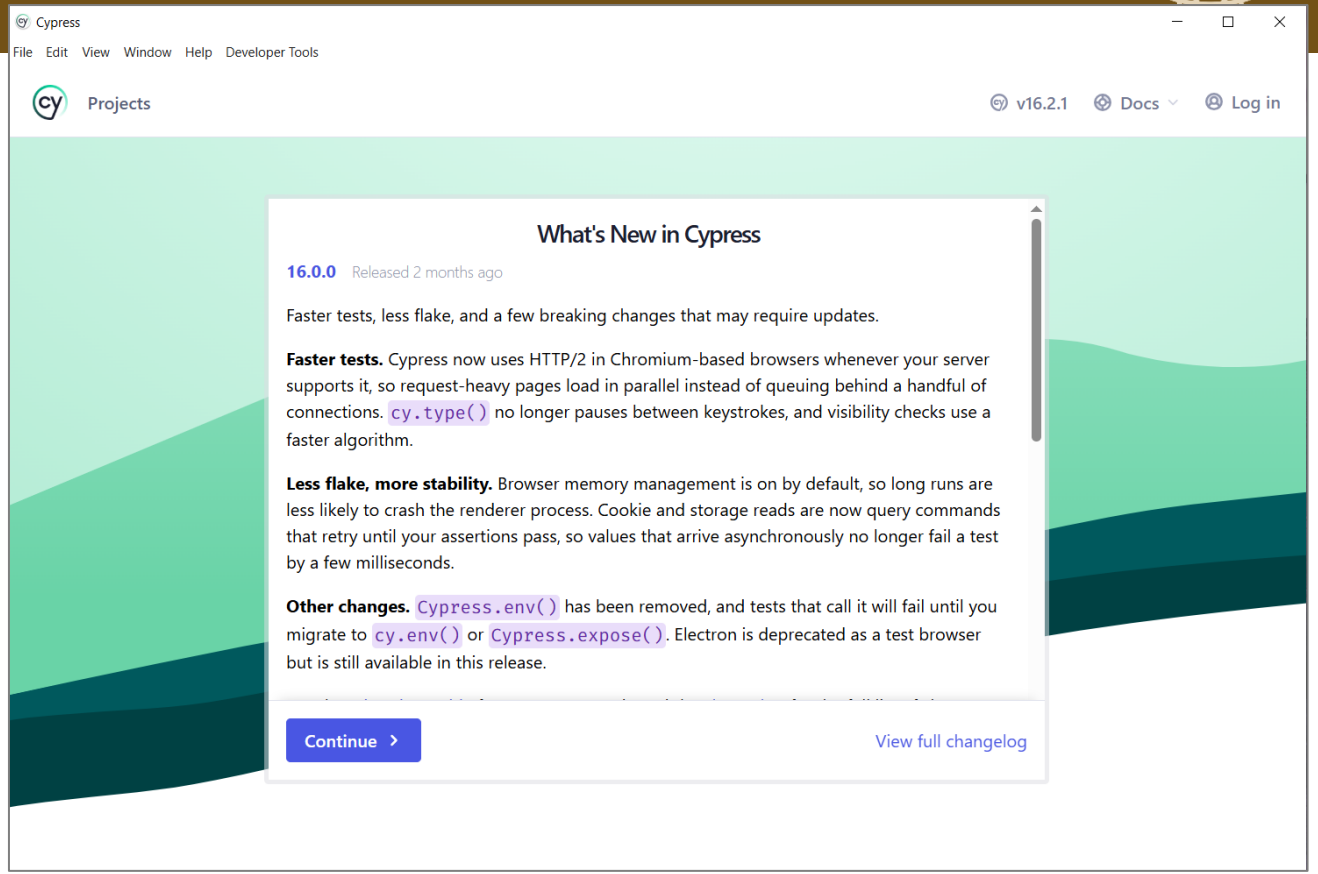
```
"devDependencies": {
  "@testing-library/dom": "^10.4.2",
  "@testing-library/jest-dom": "^7.0.1",
  "@testing-library/react": "^16.3.3",
  "@testing-library/user-event": "^14.6.8",
  "@types/jest": "^30.0.0",
  "@types/node": "^20",
  "@types/react": "^19",
  "@types/react-dom": "^19",
  "cypress": "^16.2.1",
  "eslint": "^9",
  "eslint-config-next": "16.3.0",
  "jest": "^30.5.2",
  "jest-environment-jsdom": "^30.5.2",
  "ts-jest": "^29.4.15",
  "ts-node": "^10.9.2",
  "typescript": "^5"
}
```

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "test": "jest",
  "test:watch": "jest --watch",
  "cypress": "cypress open"
},
```

Using Cypress

- Run Cypress

```
npm run cypress
```



Testing Network Requests



Spy/Stub Network Request

- Developer สามารถ spy การเชื่อมต่อ รวมไปถึง stub ส่วน backend ผ่านคำสั่ง `cy.intercept()`
  - การ Spy การเชื่อมต่อเพียงอย่างเดียว

```
cy.intercept(url)
cy.intercept(method, url)
```

  - เช่น

```
cy.intercept('/users/**')
```

```
cy.intercept('POST','/users').as('new-user')
```

```
cy.intercept({
  method: 'GET',
  url: '/users*',
  hostname: 'localhost',
})
```

  - ใช้คำสั่ง `wait` เพื่อรอการทำ Network Request เพื่อ assert

```
cy.wait('@new-user').should('have.property', 'response.statusCode', 201)
cy.get('@new-user').its('request.body').should('deep.equal',JSON.stringify({
      id: '101',
      firstName: 'Joe',
      lastName: 'Black',
    })
  )
```

Testing Network Requests



Spy/Stub Network Request

- การ Spy การเชื่อมต่อ และ Stub Response เพื่อควบคุมพฤติกรรมของ HTTP request โดยระบุ body, HTTP Status Code, headers ฯลฯ แบบ static

```
cy.intercept(url, staticResponse)
cy.intercept(method, url , staticResponse)
```

- ตัวอย่าง

```
cy.intercept('POST', '/users*’, { statusCode: 201,
  body: {
    name: 'Peter Pan',
  }})
```

- ตัวอย่าง Dynamic Stubbing

```
cy.intercept('/users*', { hostname: 'localhost' },
(req) => {
  /* do something with request and/or response */
})
```

Testing Network Requests



- **Fixture**
  - ไฟล์ข้อมูลที่ import เข้ามายัง Cypress test
  - โดย default นิยมสร้างไว้ใน `/cypress/fixtures/fileName.json`
  - ตัวอย่าง Stub Response ด้วยข้อมูลใน fixture

```
cy.intercept('/users.json', { fixture: 'users.json' })
```

```
cy.intercept('GET', '/users', (req) => {
  req.reply({
    statusCode: 200, // default
    fixture: 'users.json'
  })
})
```

Testing Network Requests in Next.js 13



- Caution 1 : Server Component / SSR
  - `cy.intercept` อาศัย in-browser API ในการตรวจจับ request เพื่อ intercept
  - Request จาก SSR ใน Next.js ไม่ได้มาจาก web browser ทำให้ `cy.intercept` ไม่รู้จัก
  - `cy.intercept` จึงไม่รองรับการ intercept call เพื่อ stub data fetching calls ที่เกิดบน server-side

Testing Network Requests in Next.js 13



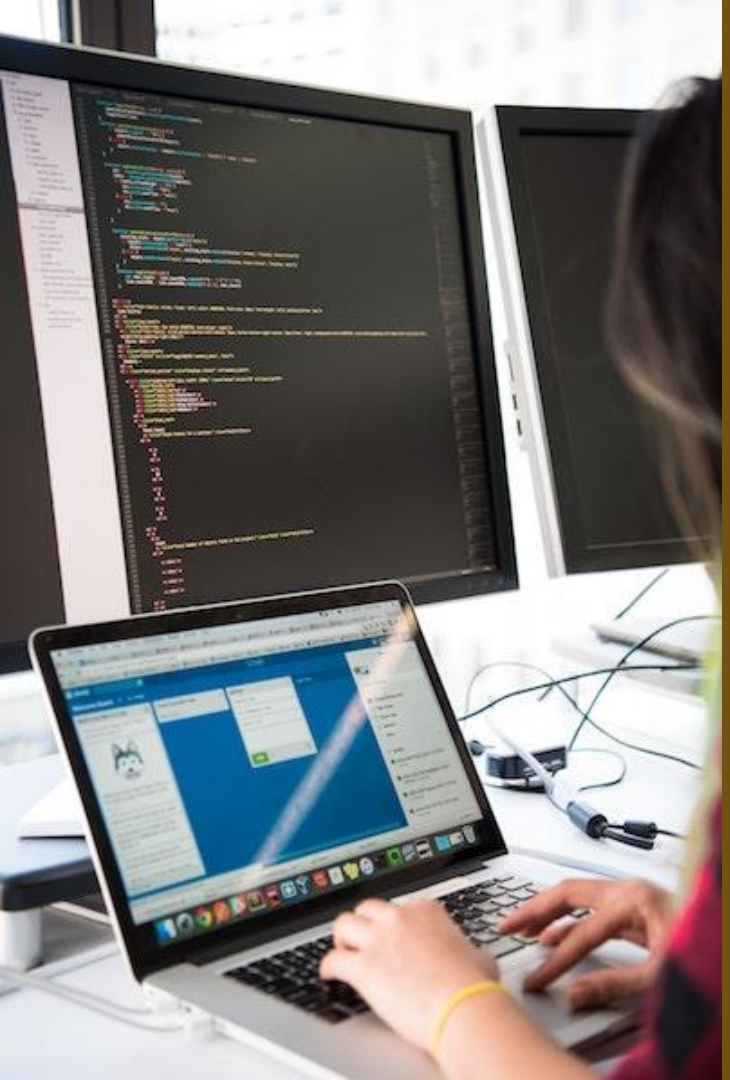
- **Caution 2: Cached Response**
  - หาก Response ที่ได้รับมาเป็น Response ที่มาจาก Cache จะทำให้ Response ที่ได้รับใน `Cy.wait()` เป็น
    - `Status Code 304 (Not Modified)` เพื่อระบุว่า response ใน cache ยังคง valid อยู่
    - Response Body จะ empty

```
httpVersion: "1.1"
method: null
statusCode: 304
statusMessage: "Not Modified"
```

  - หากต้องการ Test ค่า Response จริงจาก Server ต้องหลีกเลี่ยงการใช้ cached response
    - Server จะส่ง data จริงถ้า server ไม่สามารถระบุได้ว่า client มี data อยู่แล้ว โดยพิจารณาจาก data "cache key" ใน `if-none-match` request header
    - ต้องลบ `if-none-match` ออกจาก request

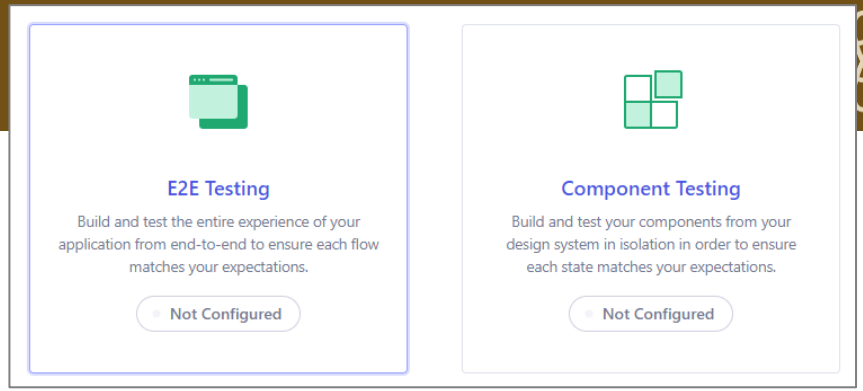
```
▼ request:
    body: ""
  ▼ headers:
      accept: "application/json, text/plain, */*"
      accept-encoding: "gzip, deflate, br"
      accept-language: "en-US"
      host: "localhost:3000"
      if-none-match: "W/"13e-aKf4C8F/V4Pp9T+FL4qIaeD2+ak""
      proxy-connection: "keep-alive"
```

```
cy.intercept(url, req => {
    delete req.headers['if-none-match']
  })
```

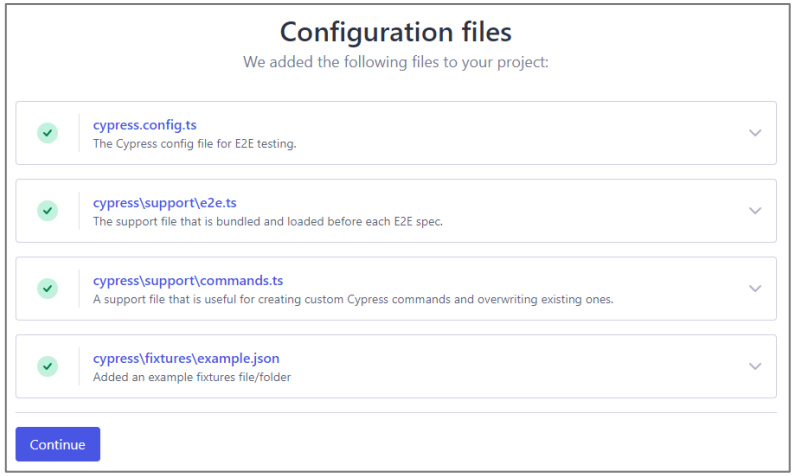


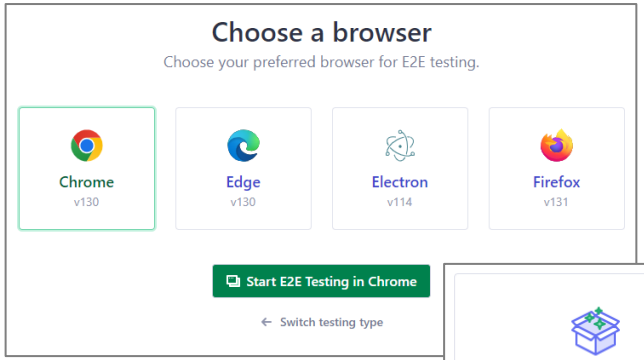
Workshop 14.2: Network Interception

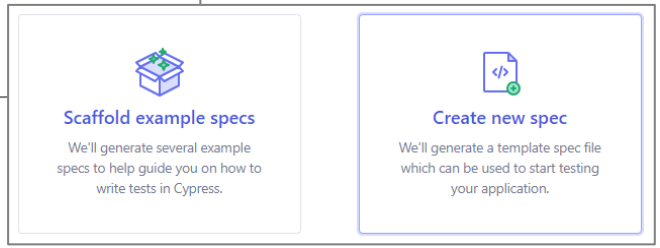
Network Interception



- สร้าง E2E Testing เพื่อทดสอบการแสดงข้อมูล Cars ของ Route `/cars`
  - Run cypress ด้วย `npm run cypress`
  - เลือก `E2E Testing`
  - ใช้ Configuration Files ที่เป็น default แล้ว **Create new spec**

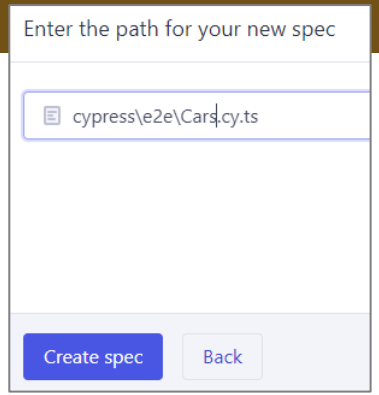


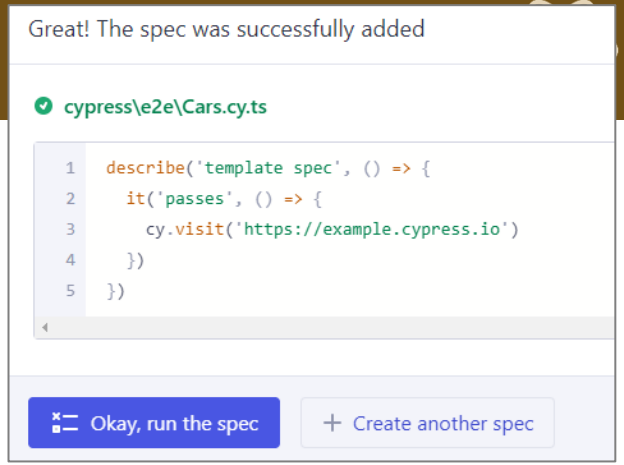




Network Interception

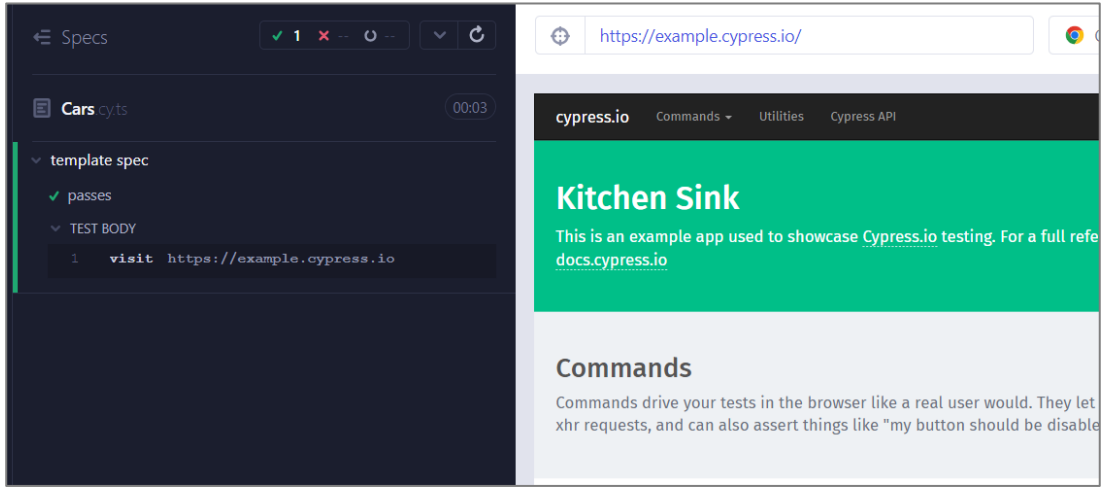
- สร้างไฟล์ Spec ไว้ที่ `cypress\e2e\Cars.cy.ts`
  - สังเกต Code ตั้งต้น เป็นการ visit link
  - เมื่อกด run the spec จะเป็นการ visit link ดังกล่าว







- ใน VS Code Terminal กด `Ctrl + c` เพื่อ stop cypress run



E2E Testing



- ใน `cypress.config.ts` ที่ Root Directory
  - ตั้งค่า base URL สำหรับการ run E2E test

```
e2e: {
  setupNodeEvents(on, config) {
    // implement node event listeners here
  },
  baseUrl: 'http://localhost:3000',
},
```

- ติดตั้ง `start-server-and-test` package สำหรับ server Next.js server ที่ port 3000 เพื่อรองรับการ test

```
npm i -D start-server-and-test
```

- ใน `package.json`
  - เพิ่ม script ในการ run E2E test พร้อมเปิด cypress

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "test": "jest",
  "test:watch": "jest --watch",
  "cypress": "cypress open",
  "e2e": "start-server-and-test dev 3000 cypress"
},
```

E2E Testing

- เนื่องจาก project นี้มีการติดตั้งทั้ง jest และ cypress ซึ่งมีการใช้งาน expect ที่มี syntax แตกต่างกัน เพื่อหลีกเลี่ยง conflict
  - สร้างไฟล์ `cypress\tsconfig.json` เพื่อตั้งค่าให้ใช้ type ของ cypress สำหรับไฟล์ใน `cypress` directory

cypress > tsconfig.json > ...

```jsonc
{
  "extends": "../tsconfig.json",
  "compilerOptions": {
    "types": ["cypress", "node"]
  },
  "include": [
    "../cypress/**/*.ts"
  ]
}
```

  - สร้างไฟล์ `__tests__\tsconfig.json` เพื่อตั้งค่าให้ใช้ type ของ jest สำหรับไฟล์ใน `__tests__` directory

__tests__ > tsconfig.json > ...

```jsonc
{
  "extends": "../tsconfig.json",
  "compilerOptions": {
    "types": ["jest", "@testing-library/jest-dom", "node"],
    "skipLibCheck": true
  },
  "include": [
    "../__tests__/**/*.ts",
    "../__tests__/**/*.tsx"
  ]
}
```

E2E Testing: Fetching Data



- ใส่การ test ลงไฟล์ `cypress\e2e\Cars.cy.ts`
- Start Backend API Server

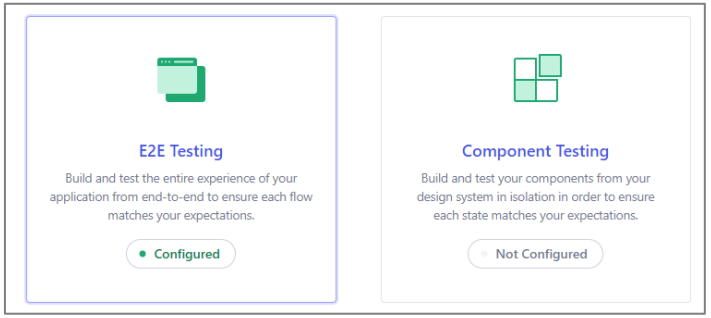
cypress > e2e > TS Cars.cy.ts > ...

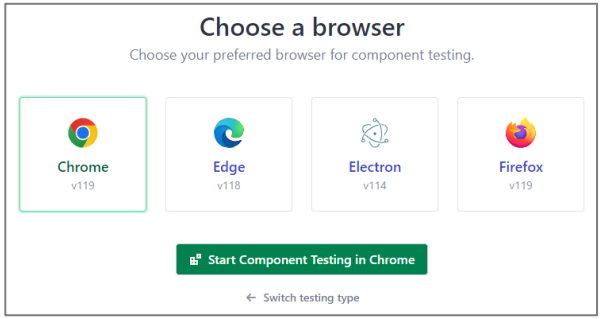
```ts
describe('Cars', () => {
  it('Should fetch cars and display', () => {
    cy.intercept({
      method: 'GET',
      url: 'http://localhost:5000/api/v1/cars'}, req => {
        delete req.headers['if-modified-since'];
        delete req.headers['if-none-match'];
        delete req.headers['proxy-connection'];
      }).as('getCars')
    // Visit Page where the API call is made
    cy.visit('/car')
    cy.wait(1000)
    cy.wait('@getCars', { timeout: 10000 }).should(({request, response})=>{
      // Assert Results
      expect(response?.statusCode).to.equal(200)
      expect(response?.body.data).to.be.not.null
      expect(response?.body.data).to.have.length(response?.body.count)
    })
  })
})
```

E2E Testing: Fetching Data

- Run Test ด้วย

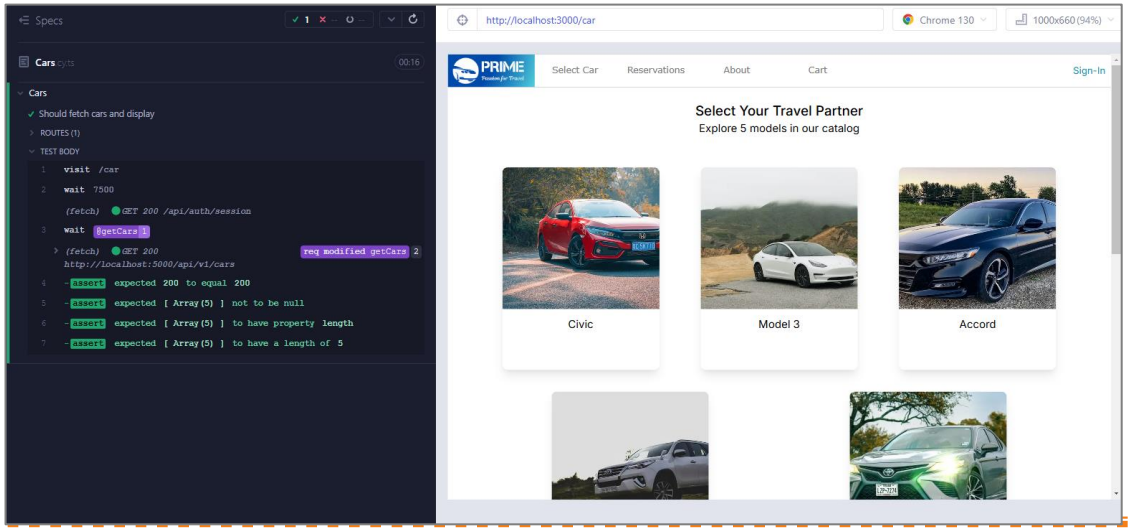
```
npm run e2e
```





- เลือก `Cars.cy.ts`





E2E Testing: Fetching Data



- เพิ่มการ Test ตาม Scenario การใช้งาน
  - User เริ่มที่หน้าแรก
    - ตรวจสอบ Menu ไปหน้า Select Car
  - User คลิกเมนูด้านบนเพื่อ Navigate ไปหน้า Select Car

```
cy.intercept({
  method: 'GET',
  url: 'http://localhost:5000/api/v1/cars'}, req => {
    delete req.headers['if-modified-since'];
    delete req.headers['if-none-match'];
    delete req.headers['proxy-connection'];
  }).as('getCars')
// Visit Homepage
cy.visit('/')
cy.wait(7500)
cy.get('a[href="/car"]').should('exist')
cy.get('a[href="/car"]').click()

cy.wait('@getCars', { timeout: 10000 }).should(({request, resp
  // Assert Results
```

E2E Testing: Fetching Data



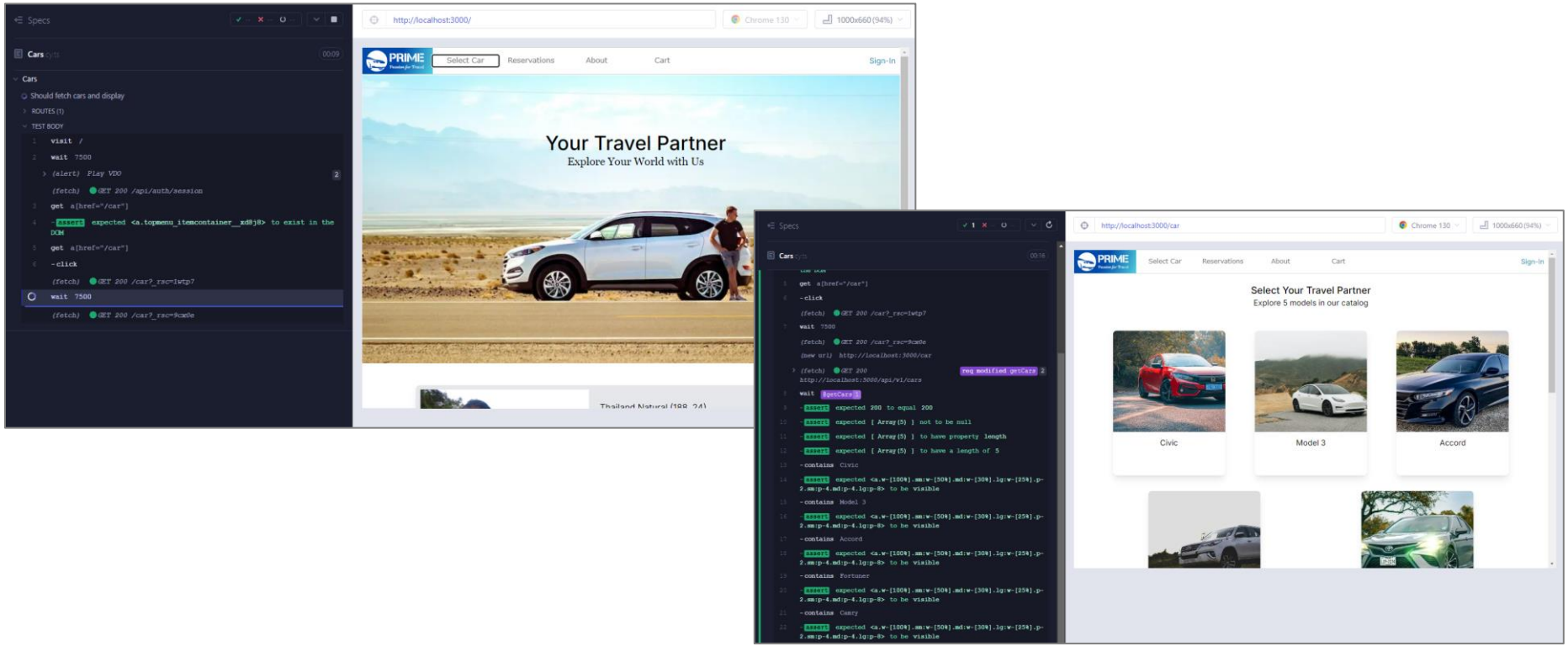
- User เข้าชมที่หน้า Select Car
  - ตรวจสอบการ fetch ข้อมูล Car
  - ตรวจสอบการแสดงผล

```
    cy.get('a[href="/car"]').should('exist')
    cy.get('a[href="/car"]').click()

    // At Car page
    var cars:string[] = []
    cy.wait('@getCars', { timeout: 10000 }).should(({request, response})=>{
      // Assert Results
      expect(response?.statusCode).to.equal(200)
      expect(response?.body.data).to.be.not.null
      expect(response?.body.data).to.have.length(response?.body.count)
      cars = response?.body.data
    }).then(()=>{
      console.log(cars)
      // Assert Display
      cars.forEach((car:any) => {
      cy.contains(car.model).should('exist')
    })
    })
  })
})
```

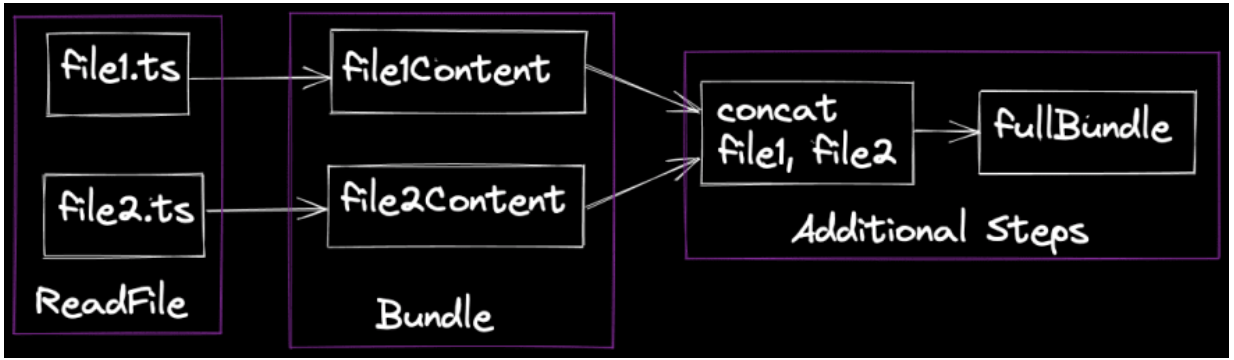
E2E Testing: Fetching Data

- ระบบจะ Run และ Automate การทดสอบ พร้อม Assert การทำงาน



Building Frontend: Module Bundler

- ปัจจุบัน Frontend Application มีความซับซ้อนมากขึ้น มากกว่าแค่ JavaScript Function ธรรมดา
  - ในช่วงการพัฒนา มีการใช้งาน Tools รวมถึง TypeScript ที่ Browser ไม่สามารถเข้าใจได้
  - นิยมใช้ Bundler ซึ่งเป็น loader รูปแบบหนึ่ง เพื่อรวม source ต่าง ๆ และจัดเตรียมให้เป็นผลลัพธ์ที่ Browser สามารถเข้าใจได้ และได้รับการ optimize ในด้านต่าง ๆ เช่น ขนาดเล็กลง หรือการทำ lazy loading เป็นต้น



https://dev.to/omher/what-is-turbopack-58ll

Building Frontend: Module Bundler



- **Webpack** เป็น bundling tool สำหรับ modern JavaScript applications ที่รวม source ต่าง ๆ (js, ts, css, scss, vue, react, angular, images) ปรับเปลี่ยนตามความจำเป็น และ bundle เข้าเป็นไฟล์ที่ browser สามารถเข้าใจได้
  - เมื่อ webpack ทำการ process application จะสร้าง dependency graph จากนั้นจึงรวม module ต่าง ๆ ที่ project ต้องการเข้าเป็น bundle(s) ซึ่งเป็น static asset ที่ใช้ในการสร้าง content
  - Next.js 11 ใช้งาน webpack 5 เป็น default
  - Next.js 13 เริ่มใช้งาน Turbopack ซึ่งเป็นรุ่นต่อมาของ webpack

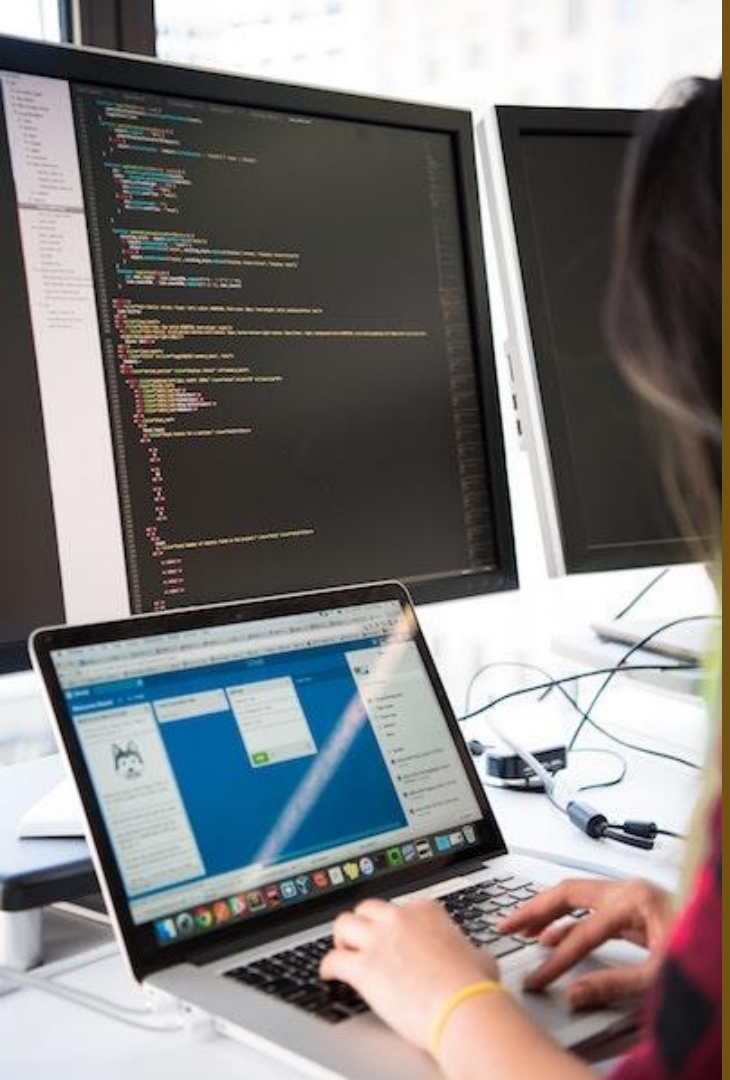
- **Turbopack** เป็น bundling tools รุ่นต่อจาก Webpack พัฒนาด้วยภาษา Rust โดยได้รับการ optimize สำหรับ JavaScript และ TypeScript
  - ทำงานได้มีประสิทธิภาพ และยืดหยุ่นกว่า webpack โดย Turbopack (alpha) ระบุว่าทำงานได้เร็วกว่า webpack ถึง 700 เท่า
  - ลองใช้งานได้โดยใช้คำสั่ง `npm run dev --turbo`

Deploy on Vercel



- แนวทางที่ง่ายที่สุดในการ deploy Next.js คือการ deploy ขึ้น Vercel platform ที่พัฒนาโดยทีมสร้าง Next.js
  - https://vercel.com

- Vercel เป็น serverless platform สำหรับ static และ hybrid application ที่สร้างขึ้นเพื่อ integrate กับ headless content, commerce หรือ database
  - สามารถใช้งานได้ Free



Workshop 14.3:
Deploy Backend on Vercel

Vercel Setup

- ใช้งาน Environment Variable แทน fix URL
  - เพิ่ม HOST Environment variable ใน config.env
  - นำไปใช้งานใน server.js

config.env

```
PORT=5000
NODE_ENV=development
MONGO_URI=mongodb+srv://machigaro:UhH5
retryWrites=true&w=majority
JWT_SECRET=+AB44KcrVw8mTutUuI46CiBH/iC
H9D2mYJdkk3+8esAOtOH4T2/pslV/
4zWVI1SLtrK66haCL1TUsazqUOvZFb6vE6gM2D
eY0BKNNGAqa6Q==
JWT_EXPIRE=365d
JWT_COOKIE_EXPIRE=365
AUTH_EMAIL=machigar@gmail.com
HOST=http://localhost
```

server.js

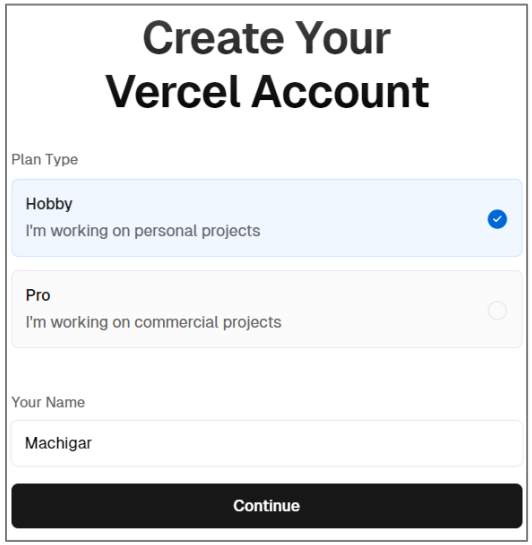
```
const PORT = process.env.PORT || 5001;
const server = app.listen(
  PORT,
  console.log(
    "Server running in",
    process.env.NODE_ENV,
    "on " + process.env.HOST + ":" + PORT
  )
);

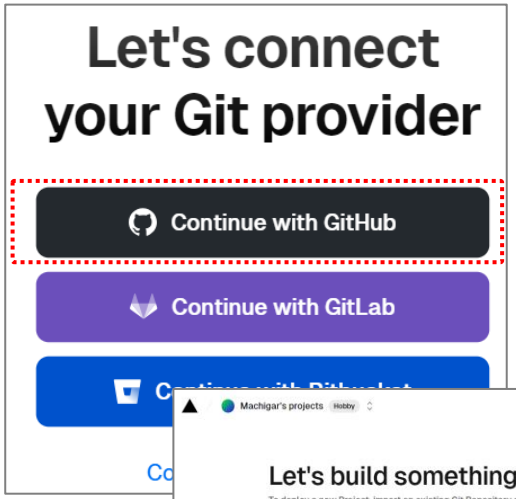
const swaggerOptions={
  swaggerDefinition:{
    openapi: '3.0.0',
    info: {
    title: 'Library API',
    version: '1.0.0',
    description: 'Car Booking API'
    },
    servers: [
      {
      url: process.env.HOST + ':' + PORT + '/api/v1'
      }
    ],
```

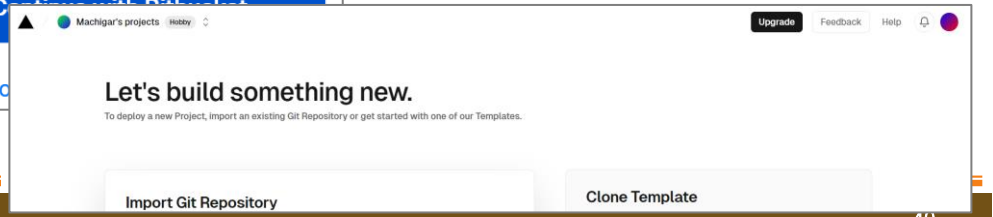
Vercel Setup

Vercel รองรับการเชื่อมต่อกับ GitHub และการ Deploy Project บน GitHub ไปยัง Vercel

- Sign-Up / Sign-In ที่ GitHub https://github.com/
- สมัครใช้งาน Vercel ที่ https://vercel.com/
  - เลือก Sign-Up และ Connect กับ GitHub เพื่อ Log-In เข้า Console ของ Vercel

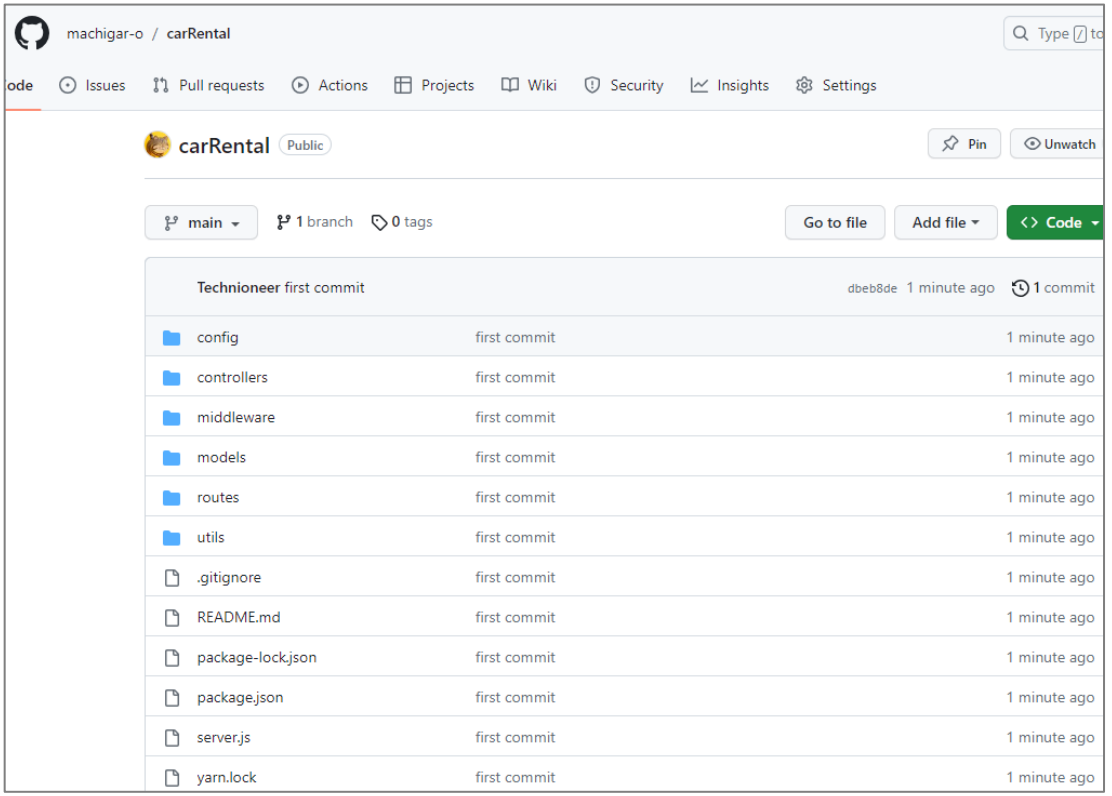






Backend Deployment

- สร้าง Git Repository สำหรับ Backend API บน GitHub
- Push Backend Project ขึ้นบน GitHub



Backend Deployment



- สร้างไฟล์ `vercel.json` ที่ Root Directory ของ Project

vercel.json > ...

```
{
    "version": 2,
    "name": "car-rental",
    "builds": [
        {
            "src": "server.js",
            "use": "@vercel/node"
        }
    ],
    "routes": [
        {
        "src": "/(.*)",
        "dest": "server.js"
        }
    ]
}
```

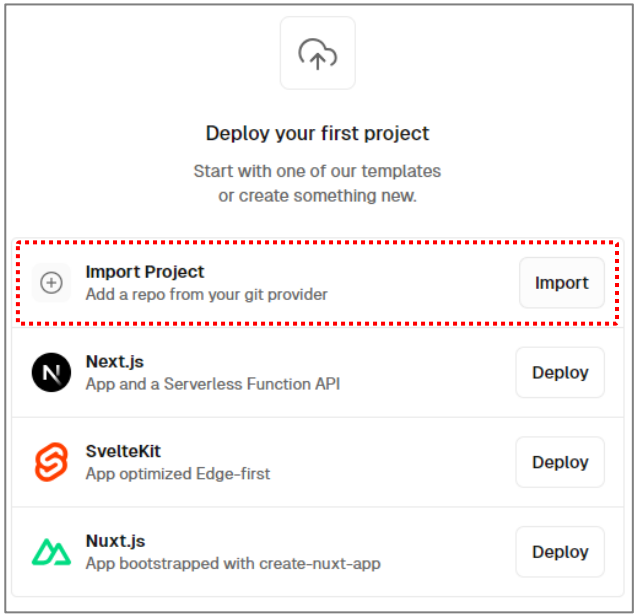
- Commit ขึ้น Git Repository

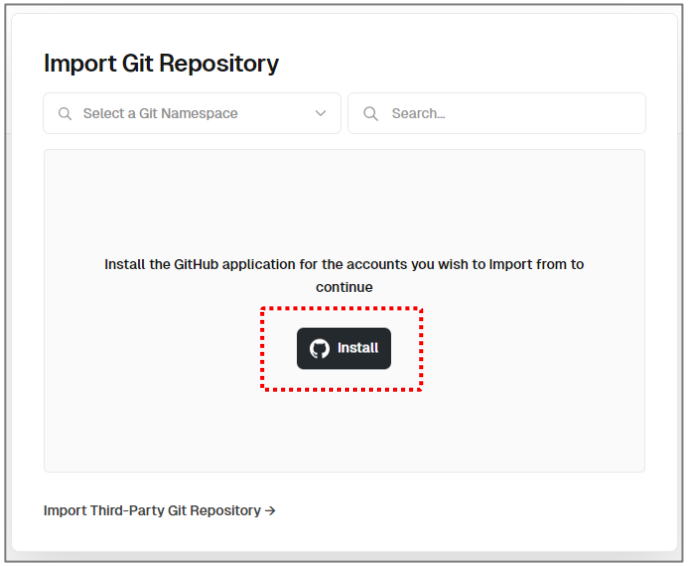
```
git add vercel.json
git commit -a -m "vercel.json added"
```

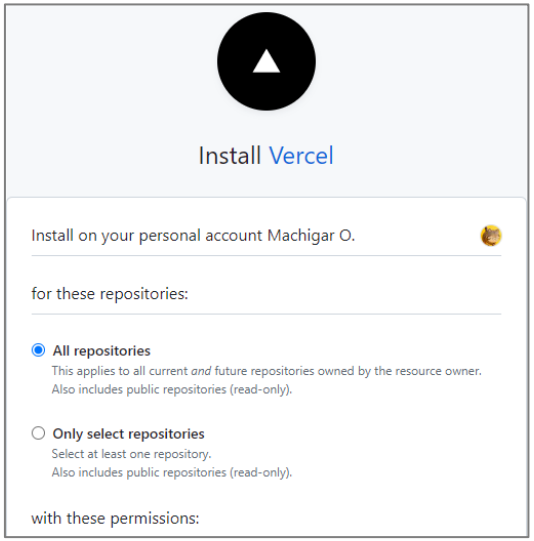
```
git push -u origin main
```

Backend Deployment

- ที่ Console ของ Vercel เลือก Import Project จาก Git Provider
  - Install GitHub application เข้าไปใน Repository ที่ต้องการนำมา import เข้า Vercel
  - เลือกทุก Repository หรือ Only Select Repositories

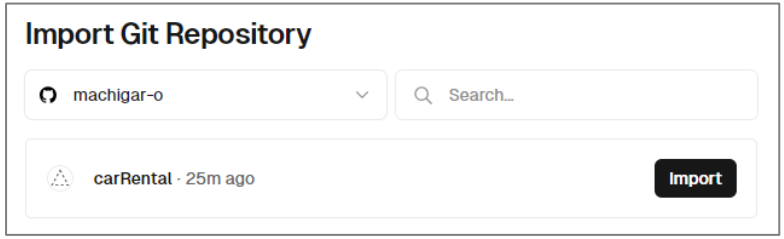






Backend Deployment

- Import carRental repository

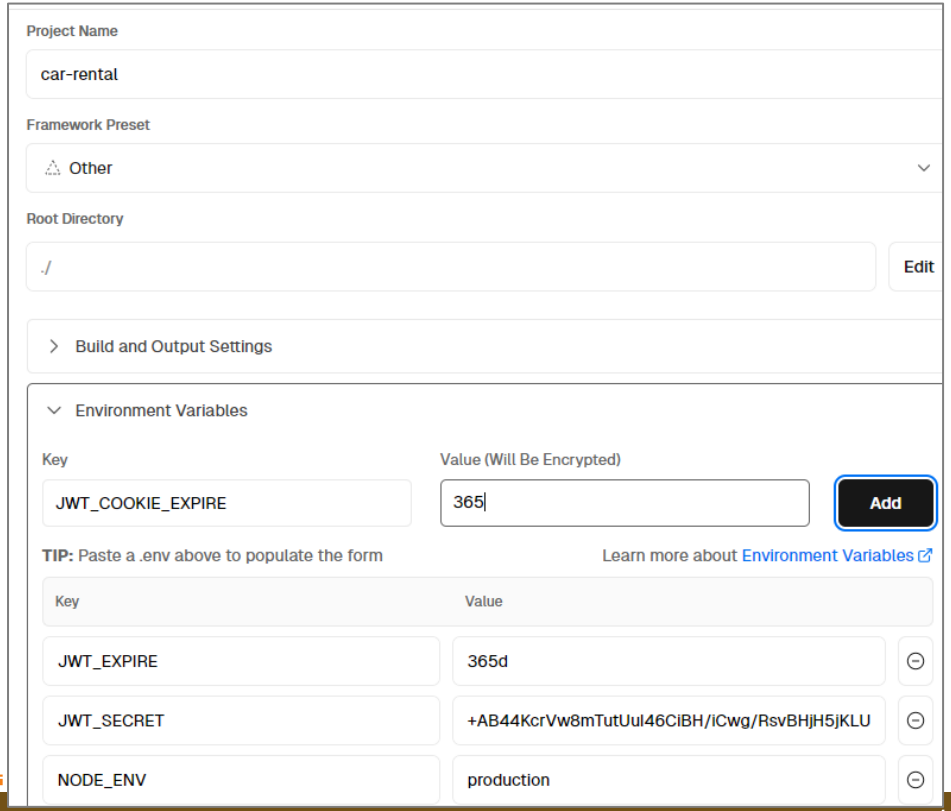


- ใส่ Environment Variable

  จาก config\config.env

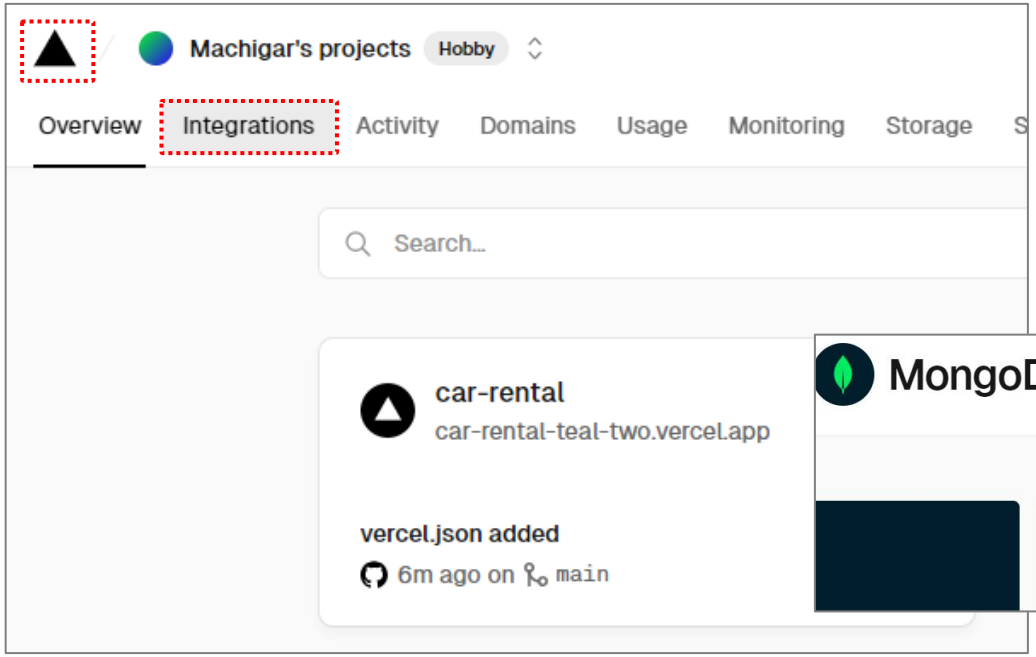
  - NODE_ENV เป็น production
  - JWT_SECRET
  - JWT_EXPIRE
  - JWT_COOKIE_EXPIRE
  - AUTH_EMAIL

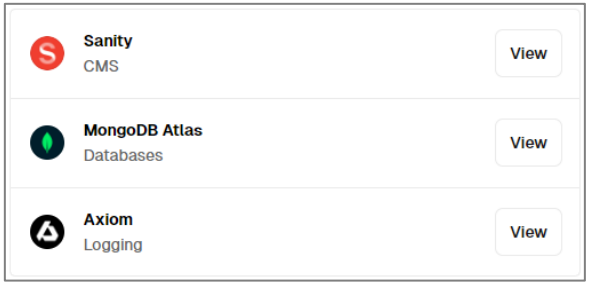
- กด Deploy

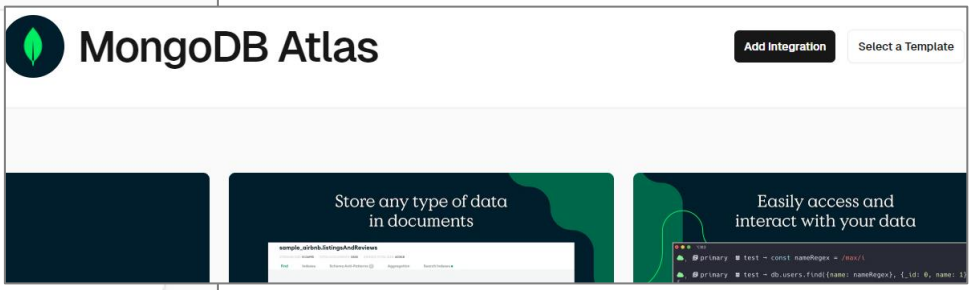


Backend Deployment

- Integrate Vercel กับ MongoDB
  - ไปที่หน้าแรก เลือก Integrations เลือก MongoDB Atlas เลือก Add Integration

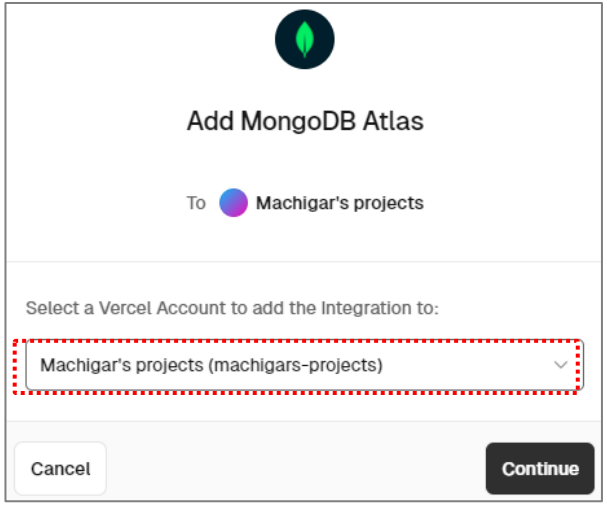


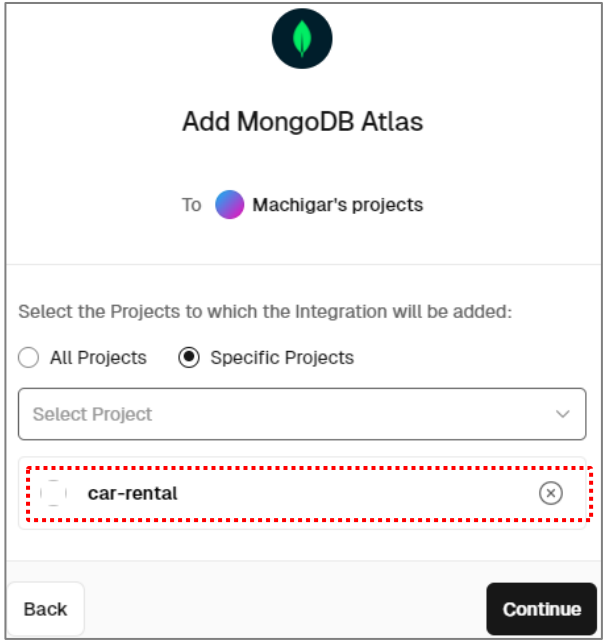


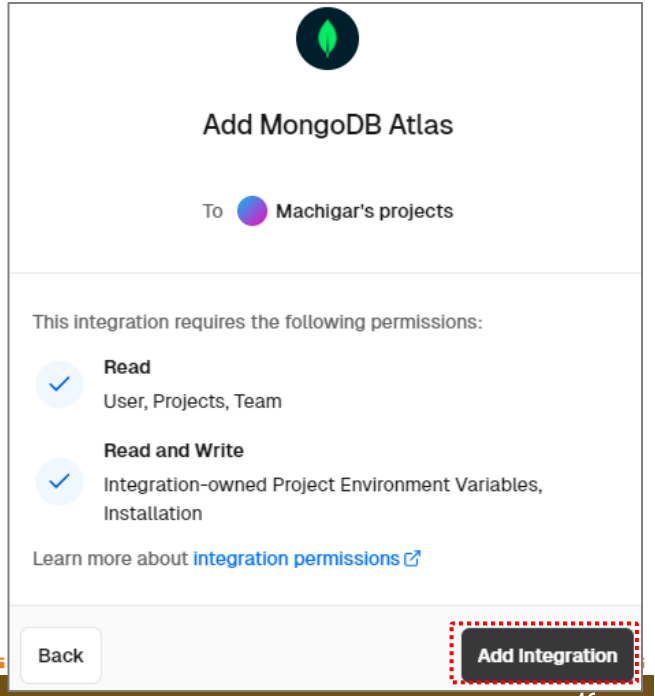


Backend Deployment

- เลือก Vercel Account ที่จะ integrate
- เลือก Project ที่จะ integrate
- Add Integration

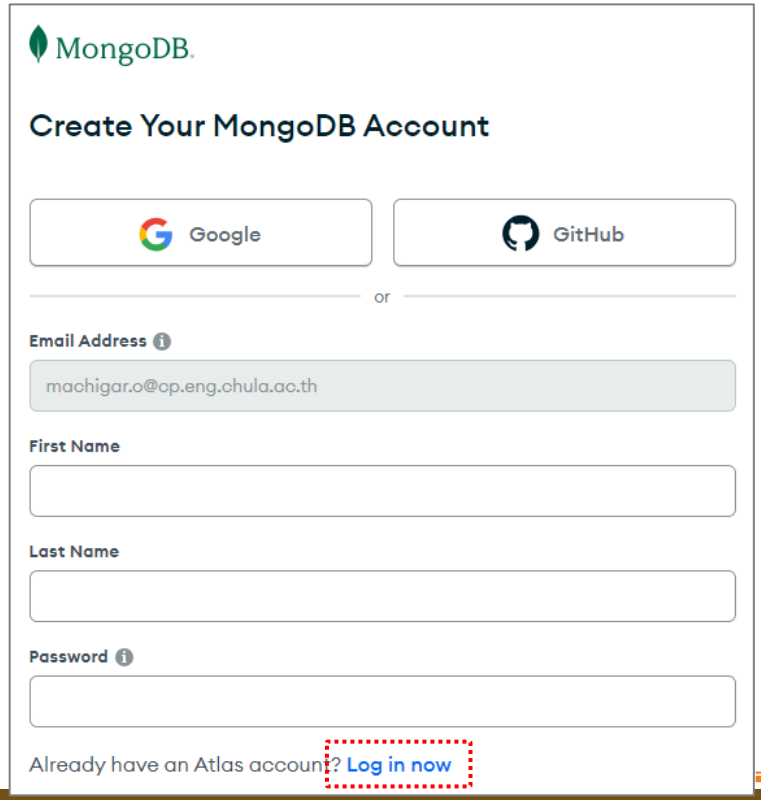


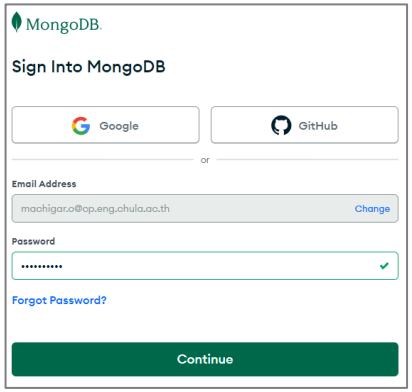


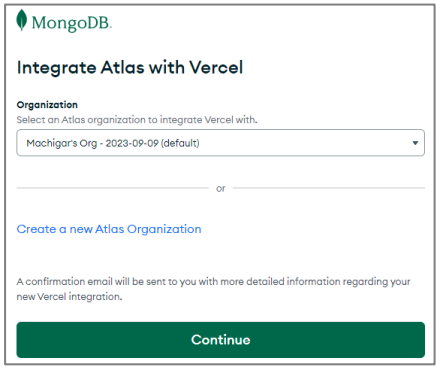


Backend Deployment

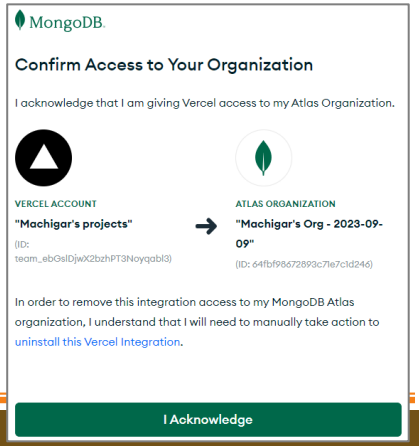
- ระบบจะ Authenticate เข้า MongoDB Atlas ให้ Log In เข้า Account MongoDB Atlas







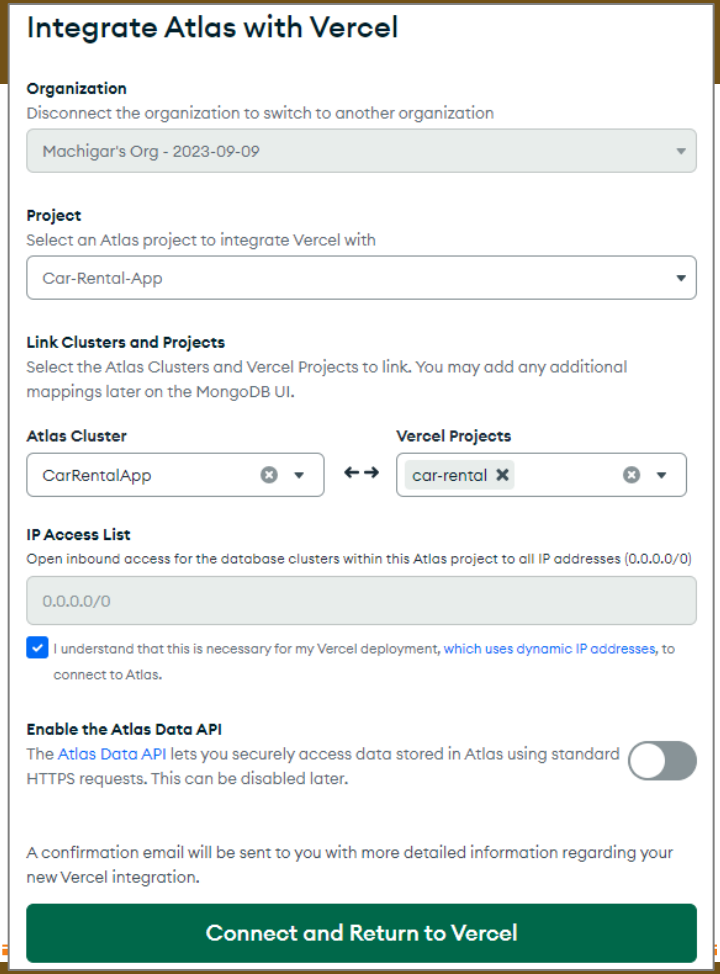
- เลือก Organization
- ยืนยันการเชื่อมต่อ



Backend Deployment

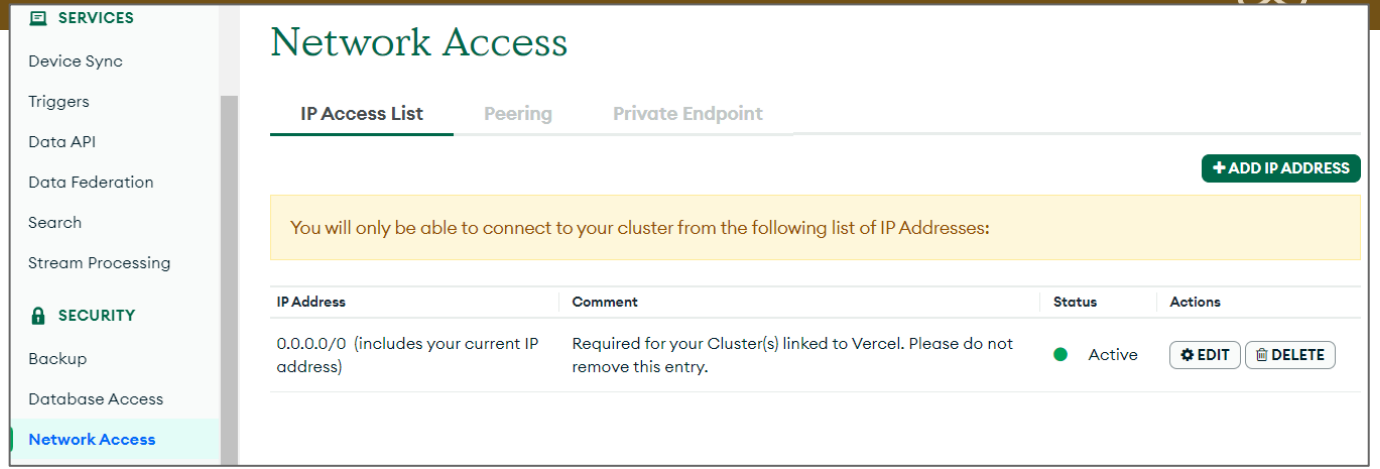
- เลือก Project ที่จะเชื่อมต่อ
- ยืนยันการเชื่อมต่อ

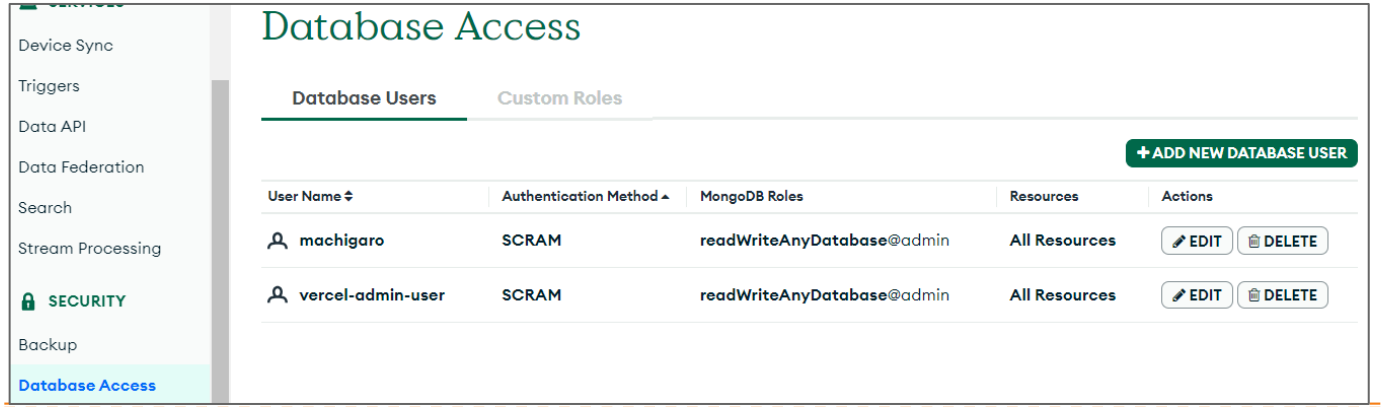
- เนื่องจาก Vercel ใช้ Dynamic IP Address การเชื่อมต่อจะเป็นการเปิดให้ทุก IP Address เข้าถึง MongoDB Atlas Project นั้นได้
  - IP Access List มี 0.0.0.0/0



Backend Deployment

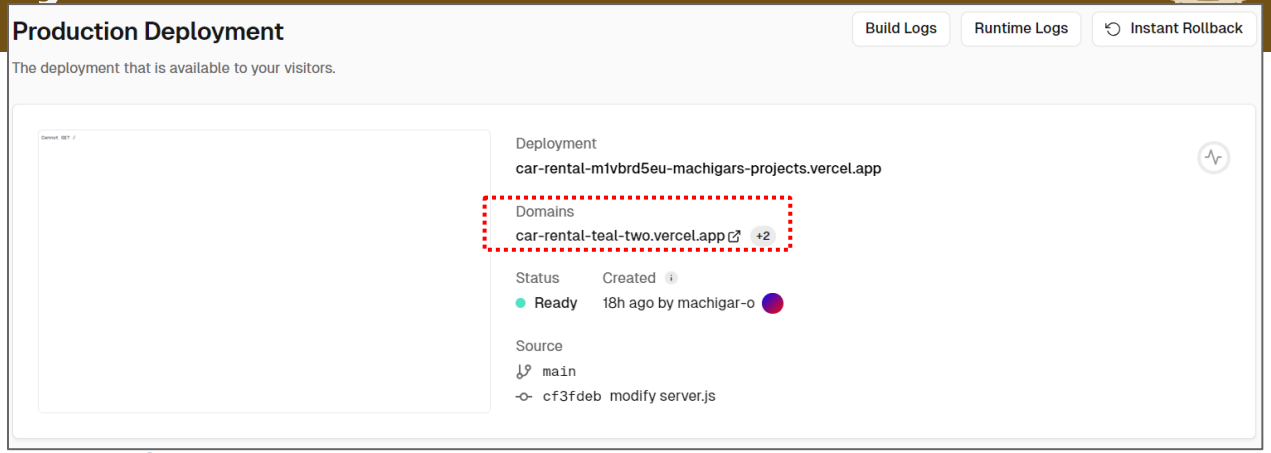
- ใน MongoDB Atlas
  - ตรวจสอบ Network Access ว่ามี 0.0.0.0/0
  - มีการเพิ่ม vercel account เข้าใน Database Access



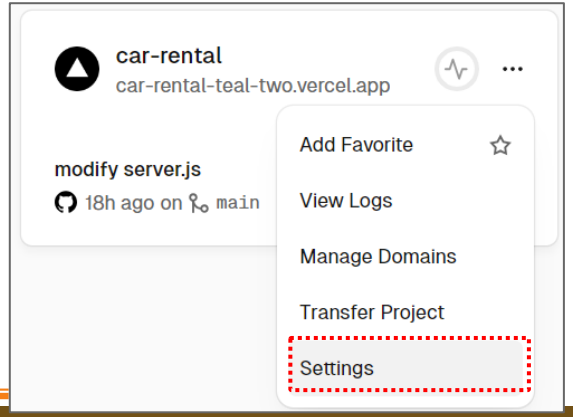


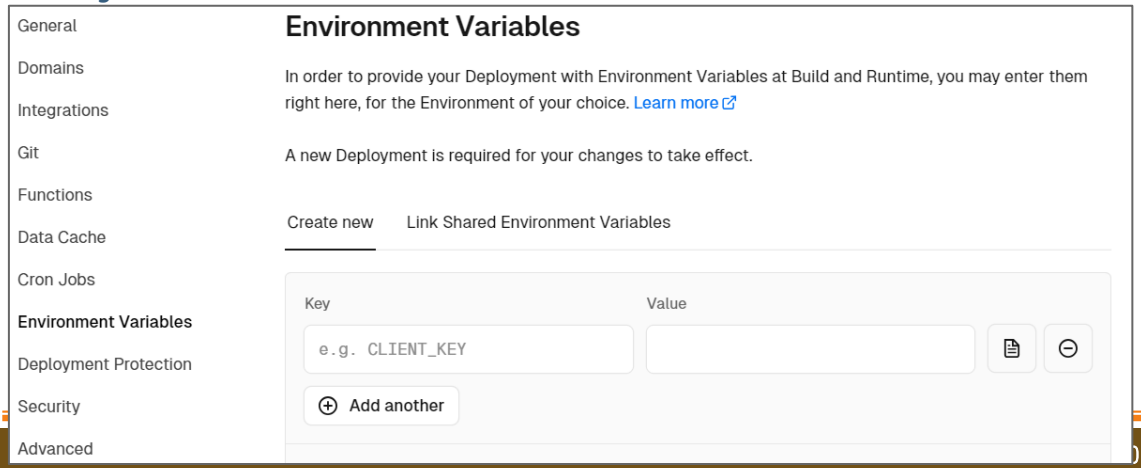
Backend Deployment

- เข้าไปใน Deployment
  - Copy Domain มาใช้



- ปรับแต่ง environment variable ที่ Project Settings

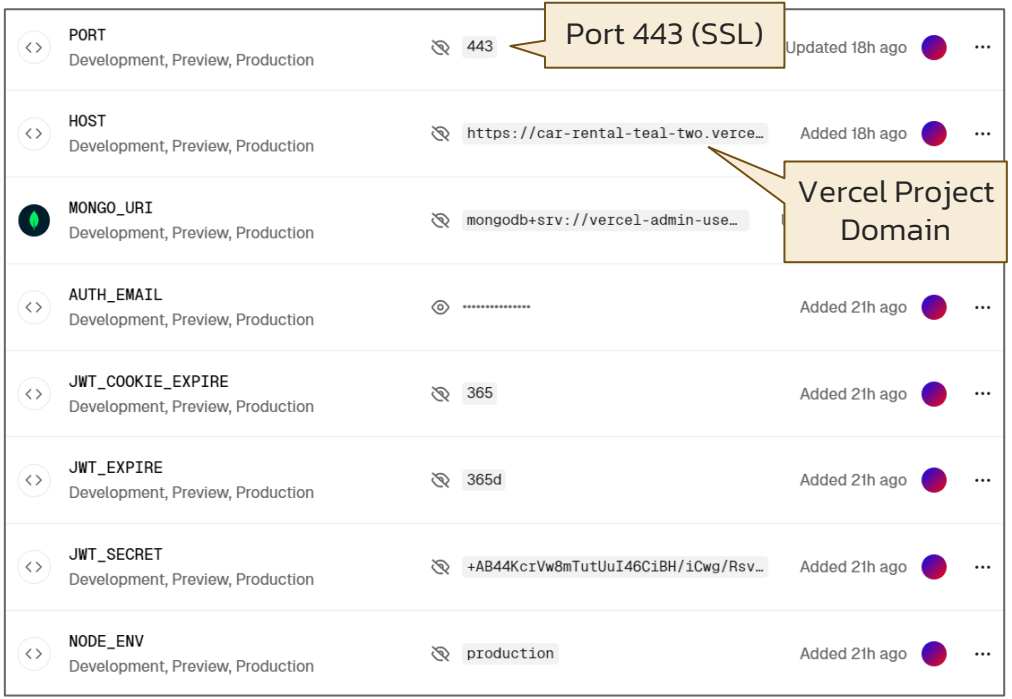




Backend Deployment

- ปรับแต่ง Environment Variable ที่เหลือ
  - Vercel Integration จะเพิ่ม MongoDB URI เข้าไปให้ ให้เปลี่ยนชื่อ Environment Variable เป็นชื่อที่เราใช้งาน คือ MONGO_URI
    - URL ที่ระบุมาให้ จะระบุชื่อ database เป็นค่า default
    - ให้ลบชื่อ database ออก
    - mongodb+srv://vercel-admin-user:*<password>*@*<projectDomain>*.mongodb.net/?retryWrites=true&w=majority

ลบชื่อ database

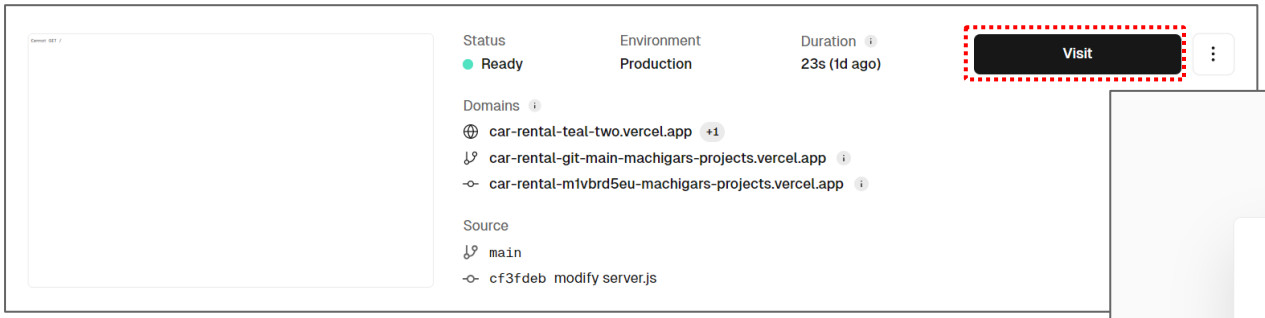


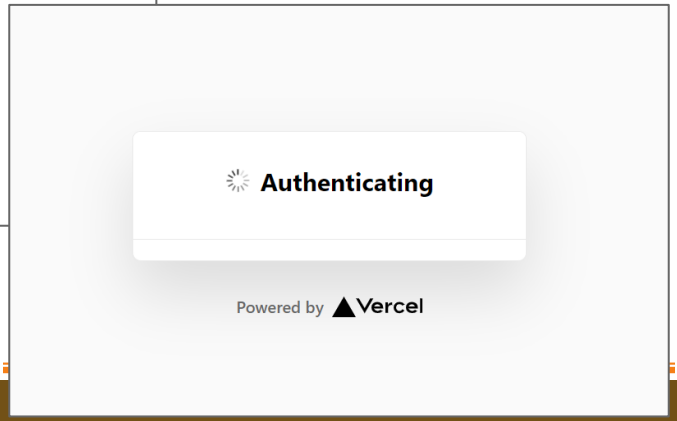
Backend Deployment

- Deployments > Redeploy



- Visit & Authenticate

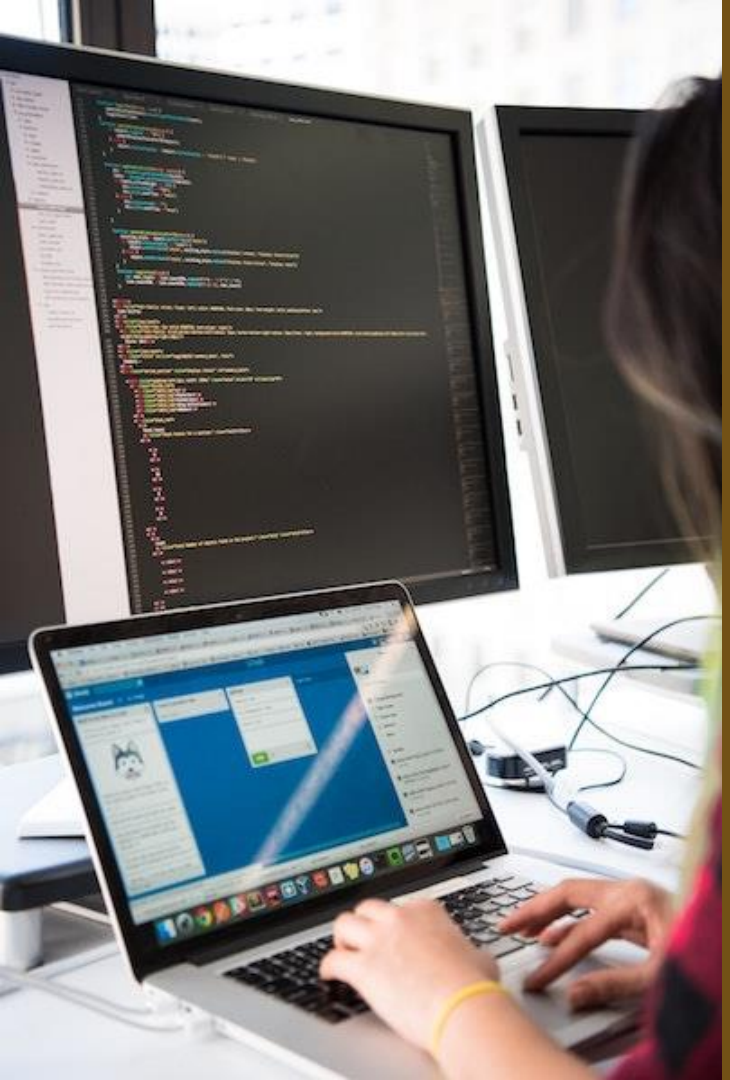




Backend Deployment

- ทดสอบโดยไปที่ https://<projectDomain>/api/v1/cars





Workshop 14.4:
Deploy Fronend on Vercel

Frontend Deployment

- การ Deploy ขึ้น Vercel ระบบขึ้นถูก Build ด้วย Strict Typing
  - เพิ่มการ typing ลงในข้อมูล Car JSON ที่เคยละเว้นไว้
  - ใน `interfaces.ts` เพิ่ม Interface
  - ใช้งานใน `CarCatalog` ไฟล์ `components\CarCatalog.tsx`

```
import Link from "next/link"
import { CarItem, CarJson } from "interfaces"

export default async function CarCatalog({carJson}: {carJson:CarJson}) {
    const carJsonReady = await carJson
    return (
        <>
        Explore {carJsonReady.count} models in our catalog
        <div style={{margin:"20px", display:"flex",
            flexDirection:"row", alignContent:"space-around",
            justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
            {
            carJsonReady.data.map((carItem:CarItem)=>(
                <Link href={`/car/${carItem.id}`}
                className="w-[100%] sm:w-[50%] md:w-[30%] lg:w-[25%]
                p-2 sm:p-4 md:p-4 lg:p-8">
                <ProductCard carName={carItem.model} imgSrc={carItem.picture}/>
                </Link>
            ))
            }
```

```
    returnDate: string
    returnLocation: string
}

export interface CarItem {
    id: string
    model: string
    picture: string
}

export interface CarJson {
    count: number
    data: CarItem[]
}
```

Frontend Deployment



- ใช้งานใน app\(carinfo)\car\page.tsx

```tsx
import { CarJson } from "interfaces"

export default async function Car() {
    const cars:CarJson = await getCars()

    return (
        <main className="text-center p-5">
            <h1 className="text-xl font-medium">Select Your Travel Partner</h1>
            <Suspense fallback={ <p>Loading ... <LinearProgress/></p> }>
            <CarCatalog carJson={cars}/>
```

- ใช้งานใน components\CarPanel.tsx

```tsx
import { CarItem, CarJson } from "interfaces"

export default function CarPanel() {

    const [carResponse, setCarResponse] = useState<CarJson|null>(null)
```

```tsx
{
    carResponse.data.map((carItem:CarItem)=>(
        <Link href={`/car/${carItem.id}`} className="w-1/5">
        <ProductCard carName={carItem.model} imgSrc={carItem.picture}
            onCompare={ (car:string)=>dispatchCompare({type:'add', carName:car})
        />
```

Frontend Deployment



- Remove Timeout/Delay ที่ไม่จำเป็น
  - ใน `libs\getCars.tsx`
    Remove การเพิ่ม 5s delay

```
export default async function getCars() {

  //await new Promise( (resolve)=>setTimeout(resolve, 5000) )

  const response = await fetch(`${process.env.BACKEND_URL}/api/v1
  if(!response.ok) {
```

- งดเว้นการ upload test components ขึ้นบน GitHun
  - เพิ่มรายการไฟล์และโฟล์เดอร์ต่อไปนี้เข้าใน `.gitignore`

```
# cypress
/cypress
src/components/*.cy.tsx
cypress.config.ts

#jest
/__tests__
jest.config.js
jest.setup.js
```

Frontend Deployment



- ปรับเปลี่ยน URL ให้กำหนดโดย Environment Variable
  - ใน `.env.local` เพิ่ม Environment Variable เก็บ URL ไปยัง Frontend และ Backend API

```
$ .env.local
1   NEXTAUTH_URL=http://localhost:3000
2   NEXTAUTH_SECRET='Ey7nTKnggBc0bRN8WUjy
3   MONGO_URI=mongodb+srv://machigaro:UhK
    retryWrites=true&w=majority
4   FRONTEND_URL=http://localhost:3000
5   BACKEND_URL=http://localhost:5000
```

  - ใน `next.config.js` ทำการ expose Environment Variable มายัง JavaScript Bundle สำหรับ Client Component

```
    },
    experimental: {
        serverActions: true
    },
    env: {
        FRONTEND_URL: process.env.FRONTEND_URL,
        BACKEND_URL: process.env.BACKEND_URL
    },
}
```

Frontend Deployment

- ปรับเปลี่ยน URL ให้กำหนดโดย Environment Variable
  - ใน `libs\getCars.tsx`, `libs\getCar.tsx`, `libs\getUserProfile.tsx`, `libs\userLogIn.tsx` ใช้ Environment Variable แทน URL String

```
await new Promise( (resolve)=>setTimeout(resolve, 5000) )

const response = await fetch(`${process.env.BACKEND_URL}/api/v1/cars`,
if(!response.ok) {
```

```
export default async function getCar(id:string) {
    const response = await fetch(`${process.env.BACKEND_URL}/api/v1/cars/${id}`)
    if(!response.ok) {
        throw new Error("Failed to fetch car")
    }
```

```
ort default async function getUserProfile(token:string) {

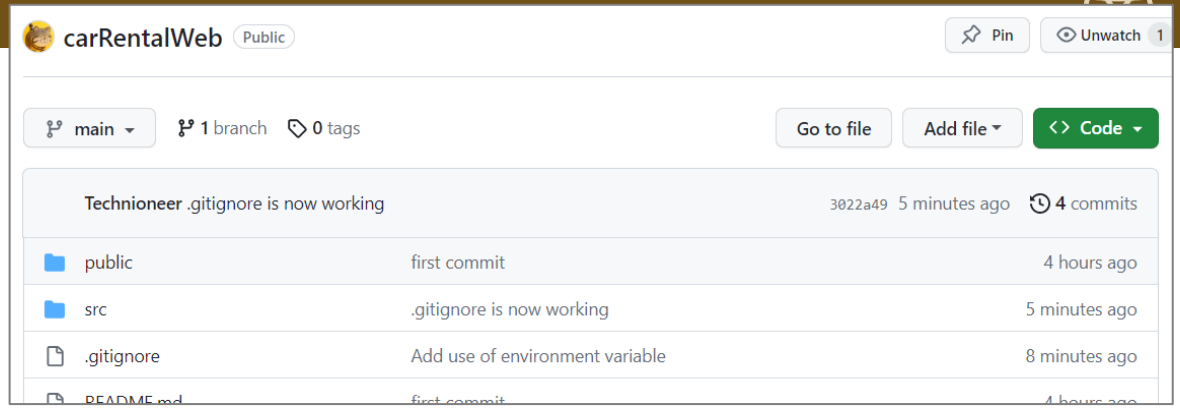
 const response = await fetch(`${process.env.BACKEND_URL}/api/v1/auth/me`
    method: "GET",
    headers: {
```

```
ort default async function userLogin(userEmail:string, userPassword:string) {

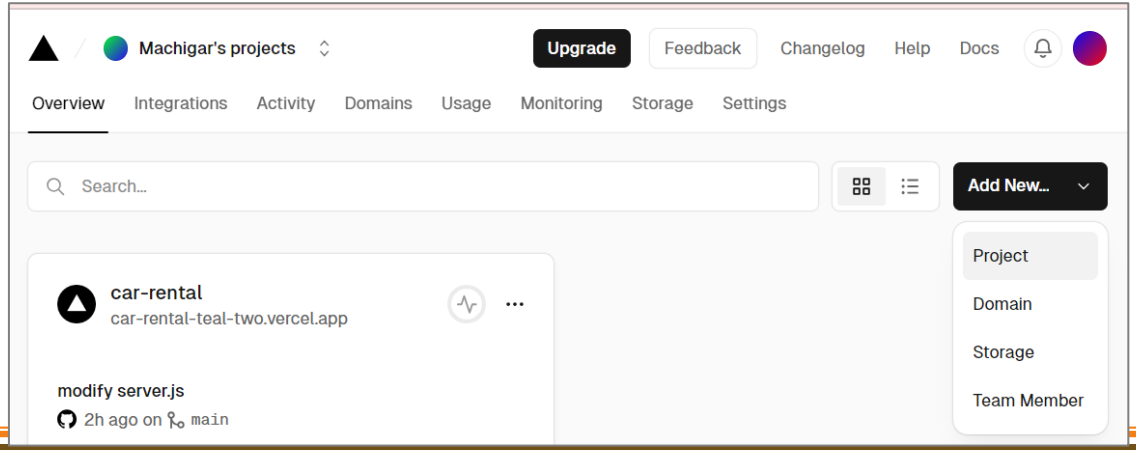
const response = await fetch(`${process.env.BACKEND_URL}/api/v1/auth/login`, {
    method: "POST",
```

Frontend Deployment

- สร้าง Git Repository สำหรับ CarRentalApp บน GitHub
- Push CarRentalApp Project ขึ้นบน GitHub

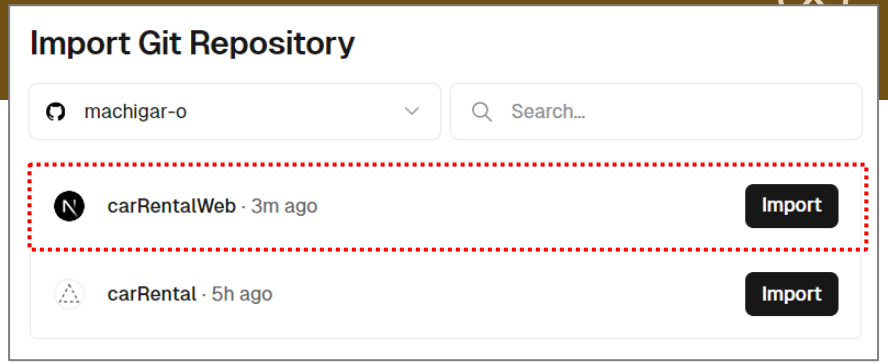


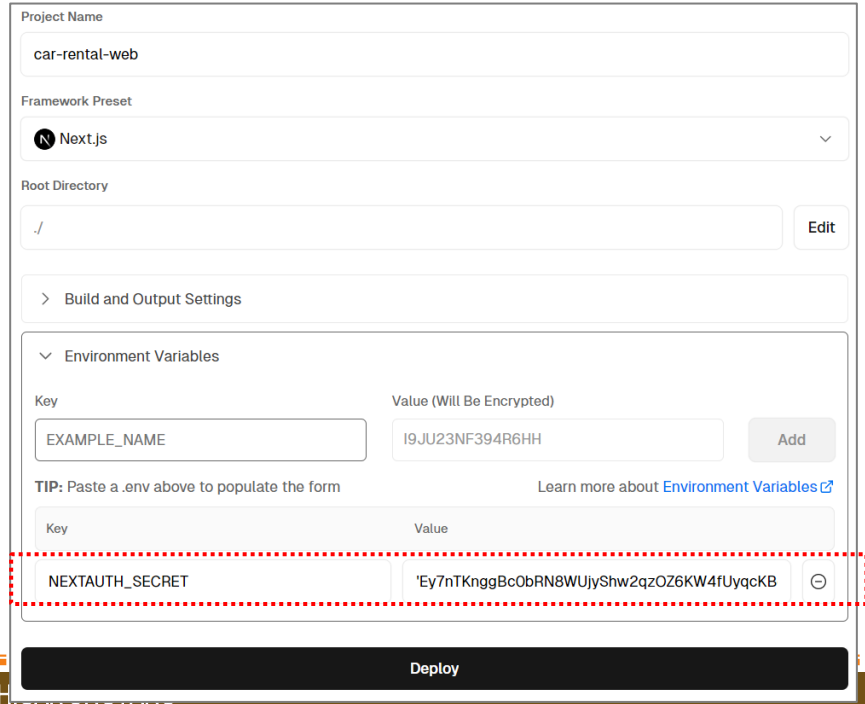
- ที่ Vercel ให้ Add New ...
  > Project



Frontend Deployment

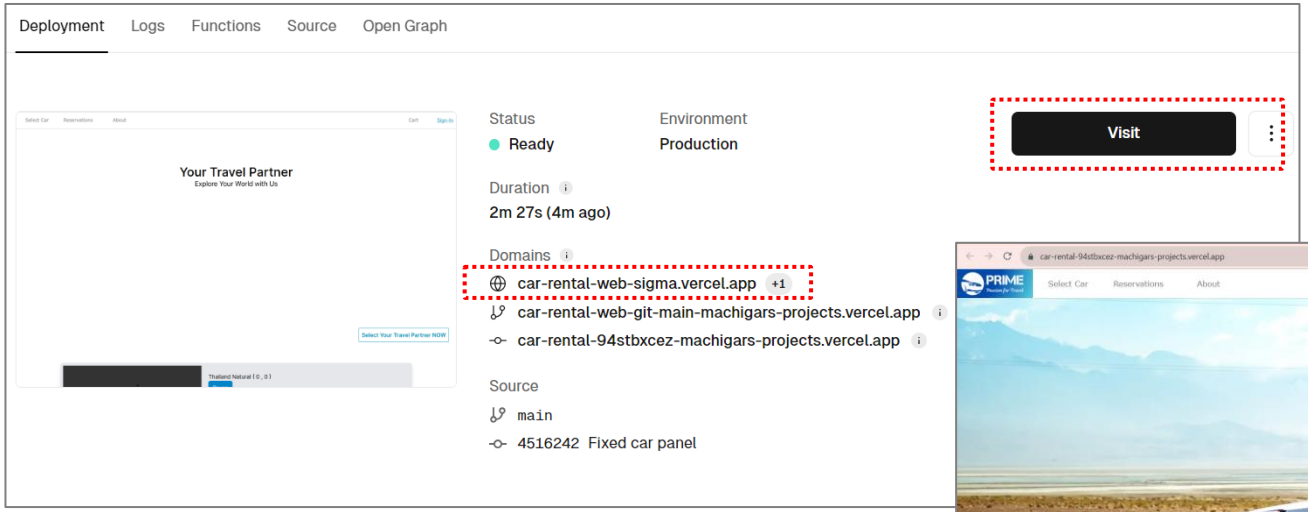
- Import จาก GitHub Repository
- Set Environment Variable
- Deploy Project

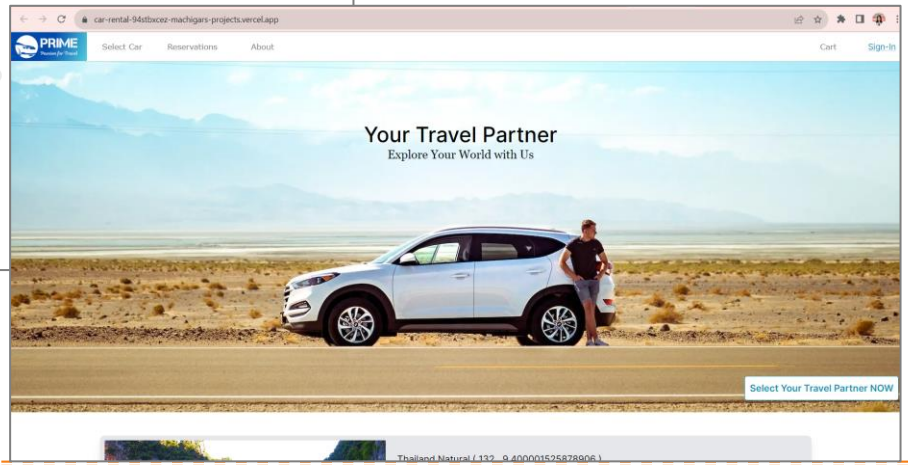




Frontend Deployment

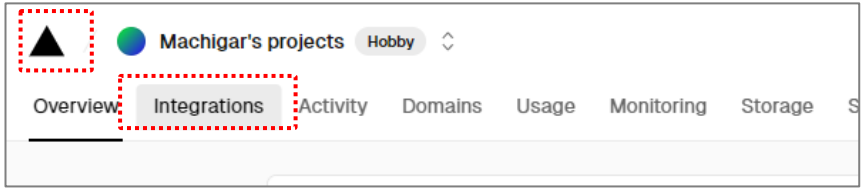
- ระบบจะได้รับการ Deploy ขึ้นมาใน Vercel และมีการกำหนด domain ให้
  - Copy Domain ไว้สำหรับใช้ตั้ง URL ใน Environment Variable





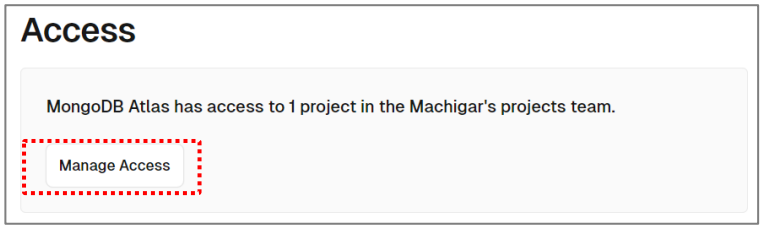
Frontend Deployment

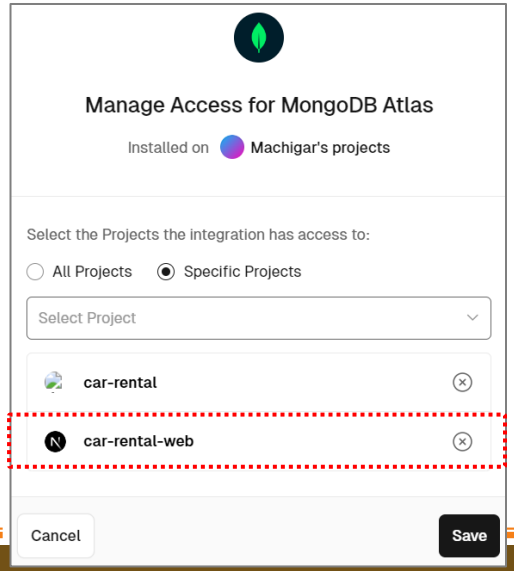
- Integrate Vercel กับ MongoDB เพื่อรองรับ Server Action
  - ไปที่หน้าแรก เลือก Integrations
  - Manage MongoDB Atlas Integration ที่มีอยู่เดิม





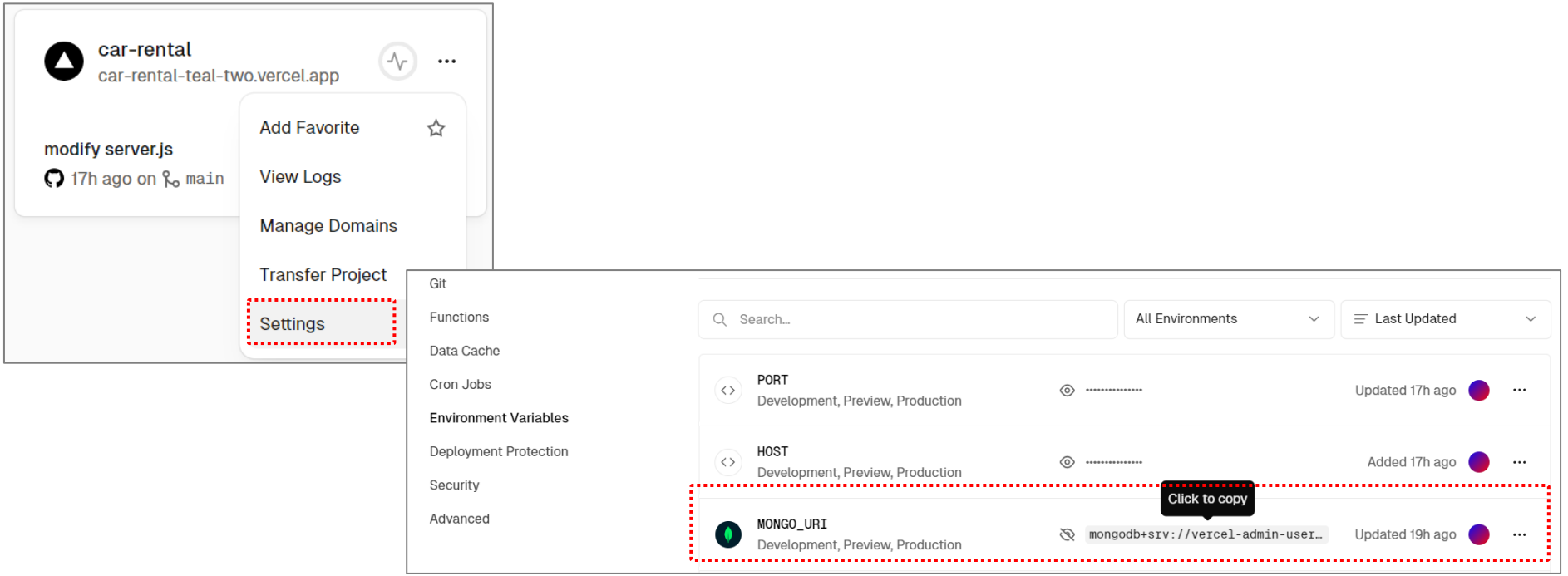
- Manage Access เพิ่ม car-rental-web project





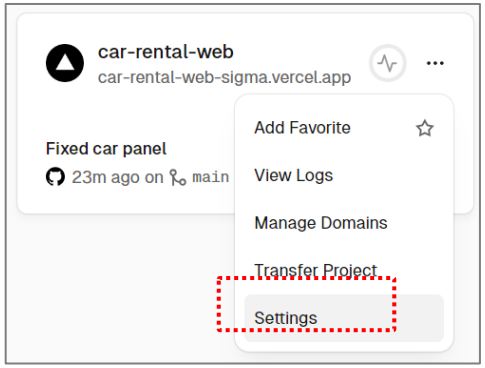
Frontend Deployment

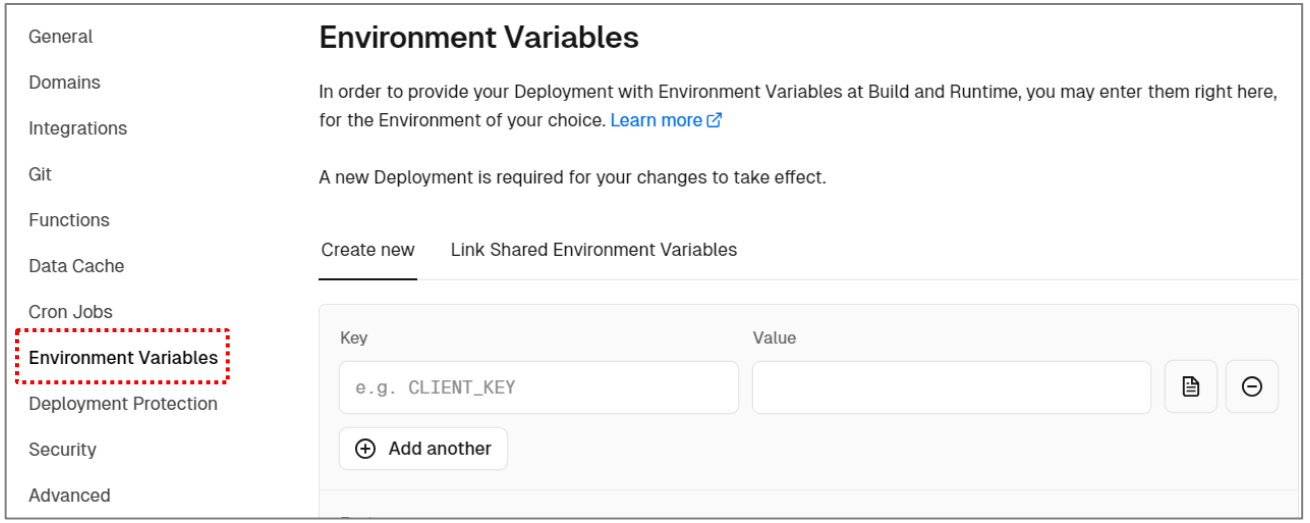
- Copy MongoDB Connection String มาจาก Backend API Project
  - Project Settings > Environment Variables



Frontend Deployment

- ปรับแต่ง Environment Variable ของ Project จาก Project Settings





- ตั้ง `MONGO_URI` Environment Variable

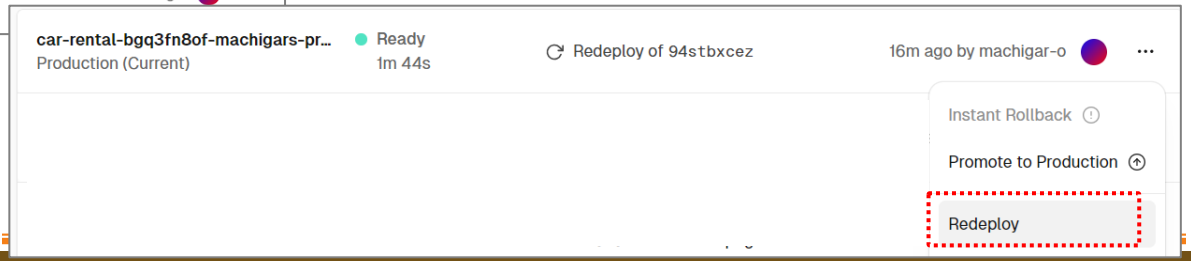


Frontend Deployment

- ตั้ง Environment Variable
  - NEXTAUTH_URL=*Domain_of_Frontend_Project*
  - FRONTEND_URL= *Domain_of_Frontend_Project*
  - BACKEND_URL= *Domain_of_Backend_Project*



- Re-Deploy Project



Frontend Deployment

- ทดสอบการทำงาน

